

XAI-APT: Explainable LLM-Based Advanced Persistent Threats Detection via Logs Embeddings and Autoencoders

Waleed Khan Mohammed^a, Hezerul Abdul Karim^{b,*}, Vik Tor Goh^c and Nouar AIDahoul^{d,*}

^aFaculty of Artificial Intelligence and Engineering, Multimedia University, Persiaran Multimedia, Cyberjaya, 63100, Selangor, Malaysia

^bCentre for Image and Vision Computing, COE for Artificial Intelligence, Faculty of Artificial Intelligence and Engineering, Multimedia University, Persiaran Multimedia, Cyberjaya, 63100, Selangor, Malaysia

^cCentre for Big Data and Blockchain Technologies, COE for Advanced Cloud, Faculty of Artificial Intelligence and Engineering, Multimedia University, Persiaran Multimedia, Cyberjaya, 63100, Selangor, Malaysia

^dComputer Science Division, New York University, Abu Dhabi, UAE

ARTICLE INFO

Keywords:

Anomaly Detection, Autoencoders, Embeddings, Explainable AI, Large Language Models, Semantic Representations, SHAP.

ABSTRACT

Advanced Persistent Threats (APTs) are sophisticated, long-term cyberattacks that are difficult to detect because they hide within massive volumes of normal system logs. Several detection solutions have been proposed to address this problem, but they still generate too many false positives due to system noise and the misinterpretation of harmless network activity as an attack. This paper presents an unsupervised anomaly detection framework that uses Large Language Models (LLMs) to convert system logs into text embeddings, which are then analyzed by autoencoders. We systematically benchmarked 17 LLM-based embedding models and 11 autoencoder architectures across five DARPA Transparent Computing (TC) datasets (SDIR, CADETS, CLEARSCOPE, THEIA, and TRACE) and five process contexts. Our results demonstrate that selecting the correct type of log data is the most important factor for accurate detection. Specifically, network traffic logs (ProcessNetflow) were demonstrated to be the most effective at revealing these hidden attacks. We demonstrate that smaller encoder models outperform large decoder models on structured log data. Furthermore, simple deterministic autoencoders provide better detection boundaries than complex probabilistic models. To ensure the model decisions are transparent to security analysts, we introduce a two-stage Explainable AI (XAI) pipeline. First, SHAP token attribution identifies the specific log tokens, such as external IP addresses and unusual file paths, that trigger an alert. Second, a latent space ablation study empirically demonstrates that these specific tokens cause the detection. Removing suspicious tokens altered the reconstruction error, providing supporting evidence for the attribution across all our tests. This framework gives engineers a reliable, fully unsupervised, and highly explainable tool for identifying rare cyber threats.

1. Introduction

Cybersecurity threats have grown significantly in scale and sophistication over the past decade, but few attack categories present as serious a challenge as Advanced Persistent Threats (APTs). Unlike opportunistic attacks that seek immediate impact, APTs are long-term, targeted intrusions in which well-resourced adversaries, including nation-state actors and organized criminal groups, quietly infiltrate a network and maintain covert access for extended periods, sometimes months or years, before their objectives are realized [1]. Their goals typically include espionage, intellectual property theft, financial fraud, and the disruption of critical infrastructure [2]. The global cost of APT-related incidents continues to rise, with targeted sectors including government, defense, healthcare, finance, and energy consistently reporting high-severity breaches [3].

A defining feature of APT campaigns is their deliberate avoidance of detection. Rather than deploying custom malware that might trigger signature-based alerts, APT actors increasingly rely on living-off-the-land (LotL) techniques, abusing legitimate system utilities such as PowerShell, Windows Management Instrumentation (WMI), and scheduled tasks that are already present on the target machine [4]. This strategy makes APT activity nearly indistinguishable from routine administrative operations at the level of individual log events. Conventional security tools, including signature-based Intrusion Detection Systems (IDS) and antivirus software, are structurally incapable of detecting attacks that intentionally avoid matching any known signature [5]. The result is a detection gap that has shown difficult to close with rule-based and pattern-matching approaches alone.

Behavior-based anomaly detection has emerged as the most promising alternative. Rather than cataloguing known attack patterns, anomaly detection systems learn a model of normal system behavior and flag deviations from that model as potentially malicious [5]. This approach is theoretically capable of detecting novel threats that have never been seen before, including zero-day exploits and previously undocumented APT techniques. Classical anomaly detection methods, such as Isolation Forest [6], One-Class SVM (OCSVM) [7], and PCA-based reconstruction [8], have been applied to

*Corresponding author(s): Hezerul Abdul Karim, email: hezerul@mmu.edu.my, Nouar AIDahoul, email: nouar.aldahoul@nyu.edu

 m.waleedkhan.ai@gmail.com (W.K. Mohammed); hezerul@mmu.edu.my (H.A. Karim); vtgoh@mmu.edu.my (V.T. Goh); nouar.aldahoul@nyu.edu (N. AIDahoul)

ORCID(s): 0009-0009-5306-348X (W.K. Mohammed); 0000-0002-7613-4596 (H.A. Karim); 0000-0003-4573-9493 (V.T. Goh); 0000-0001-5522-0033 (N. AIDahoul)

network intrusion detection and log analysis with moderate success. However, these methods evaluate each log entry in isolation and therefore miss the sequential and contextual relationships across events that are the defining characteristic of multi-stage APT campaigns [1, 9].

The emergence of LLMs offers a fundamentally different approach to this problem. Pre-trained on large corpora of text, LLMs can convert any text input into dense, high-dimensional vector representations, known as embeddings, that capture not just the content of the input but its semantic and contextual meaning [10]. When system log events are expressed as natural language sentences and encoded by an LLM, the resulting embeddings carry behavioral context that raw numerical feature vectors cannot represent. This makes LLM embeddings well-suited to the APT detection problem, where the difference between normal and malicious activity is often visible only at the level of behavioral context rather than individual event content [2, 11]. Recent work has demonstrated this potential: Benabderrahmane et al. showed that combining LLM embeddings with unsupervised autoencoders on DARPA TC provenance, which demonstrated good performance, substantially outperforming classical baselines [1]. Zuo et al. further demonstrated that LLM-enriched embeddings showed good performance on system provenance data [12].

Despite these promising results, two significant barriers remain before LLM-based anomaly detection can be deployed with confidence in real Security Operations Center (SOC) environments. The first is the absence of systematic empirical guidance on model selection. Existing studies each evaluate a small, fixed set of embedding models and autoencoder architectures, typically from a single architectural family, making it difficult for practitioners to make principled choices about which configurations to deploy on APT-specific telemetry [13]. The second, and more critical barrier, is interpretability. Deep learning models produce anomaly scores without explaining what triggered them. Security analysts operating in high-pressure SOC environments cannot act confidently on alerts they do not understand, and regulatory frameworks governing critical sectors increasingly require that automated decisions be explainable and auditable [9, 14, 15]. This opacity problem, the “black box” nature of neural network anomaly detectors, represents a fundamental practical barrier to adoption that detection accuracy alone cannot overcome.

Explainability methods such as SHapley Additive exPlanations (SHAP) [16] have been applied to supervised security classifiers with interpretable input features, producing useful feature-level attribution for alert triage. However, their application to deep unsupervised anomaly detectors operating on high-dimensional LLM embeddings remains largely unexplored [9]. When the input representation is a high-dimensional dense vector produced by a transformer model, individual dimensions carry no human-readable meaning, and standard feature attribution methods cannot be directly

applied. This creates a gap between the token-level understanding that security analysts need and the latent-space anomaly scoring that LLM-based detectors produce.

This paper addresses both barriers within a single unified framework. We present a dual-pipeline explainable anomaly detection architecture that combines systematic LLM embedding benchmarking with a novel two-stage XAI attribution framework. The primary detection pipeline encodes system audit logs from the DARPA TC dataset as dense embeddings and processes them through an unsupervised autoencoder to produce continuous anomaly scores. A parallel explainability pipeline then provides two complementary forms of attribution: a Term Frequency Inverse Document Frequency (TF-IDF) [17, 18] and SHAP pipeline that assigns exact numerical impact values to individual tokens in the flagged log, and a novel Automated Multi-Stage Ablation Study that identified threat tokens shift the embedding toward benign regions.

The specific contributions of this paper are as follows:

1. **Systematic embedding model benchmarking.** We present controlled comparison of 17 pre-trained LLM embedding models across five architectural families, including encoder-based, decoder-based, large-scale, angle-optimized, and static models, evaluated under identical experimental conditions on APT-specific system telemetry from the DARPA TC dataset.
2. **Comprehensive autoencoder architecture search.** We evaluate 11 distinct autoencoder topologies, including deterministic and probabilistic variants, within the same controlled pipeline, establishing empirical guidance on which architectural properties matter most for APT log anomaly detection and why probabilistic architectures are incompatible with post-hoc XAI attribution.
3. **Dual-pipeline XAI attribution framework.** We introduce a novel two-stage explainability framework that bridges the gap between high-dimensional LLM latent space anomaly scoring and human-readable token-level attribution. The TF-IDF SHAP pipeline provides exact numerical impact values for individual log tokens. The Automated Multi-Stage Ablation Study then empirically validates that these tokens drive the LLM’s anomaly signal by measuring the reconstruction error shift produced by their systematic removal.

The remainder of this paper is organized as follows. Section 2 reviews related work on APT detection, LLM embeddings, autoencoder architectures, and explainability in security systems. Section 3 describes the full experimental methodology, including the dataset, embedding pipeline, autoencoder architecture search, and dual XAI framework. Section 4 presents and discusses the empirical results. Section 5 summarizes the findings, discusses limitations, and outlines directions for future work.

2. Related Works

This section presents the existing literature in the field of APTs, LLMs, Cybersecurity and explainability.

2.1. Advanced Persistent Threats: Characteristics and Detection Challenges

Advanced Persistent Threats represent a fundamentally different class of cyberattack compared to opportunistic intrusions. Rather than seeking immediate impact, APT actors conduct long-term, multi-stage campaigns designed to maintain persistent, covert access to a target network for extended periods, often months or years, before their objectives are realized. The typical APT lifecycle progresses through a well-documented sequence of stages: initial reconnaissance, spear-phishing or exploit-based access, privilege escalation, lateral movement, command-and-control (C2) establishment, and eventual exfiltration or sabotage [1]. Each stage is executed slowly and deliberately to minimize detection risk.

A defining feature of APT campaigns is the systematic exploitation of legitimate system utilities to carry out malicious operations, a technique widely referred to as living-off-the-land (LotL) [4]. By abusing built-in tools such as PowerShell, Windows Management Instrumentation (WMI), and scheduled tasks, adversaries generate activity that is behaviorally indistinguishable from routine administrative operations. No suspicious binaries are dropped, and no external payloads are fetched. This deliberate mimicry of normal system behavior is the primary reason signature-based detection systems, which rely on matching known attack patterns, are structurally incapable of reliably detecting APTs [2].

The scale of documented APT incidents continues to grow. Campaigns have targeted government agencies, defense contractors, healthcare institutions, and critical infrastructure operators. Threat intelligence reports from organizations including Mandiant and CrowdStrike consistently document an increase in both the frequency and sophistication of APT operations [2]. This trajectory has pushed the research community toward behavior-based detection approaches that do not require prior knowledge of specific attack signatures, and toward machine learning methods capable of identifying anomalous behavioral patterns in large volumes of system telemetry.

2.2. Anomaly Detection Methods for Cybersecurity

Anomaly detection approaches the problem of threat detection from a fundamentally different direction than signature-based systems. Rather than cataloguing known attack patterns, anomaly detection systems first learn a statistical model of normal behavior and then flag deviations from that model as potentially malicious. This makes them theoretically capable of detecting novel attacks that have never been seen before [5].

Classical anomaly detection methods include Isolation Forest, One-Class SVM (OCSVM), and PCA-based reconstruction. Isolation Forest builds an ensemble of random partitioning trees and assigns anomaly scores based on how quickly each sample is isolated [6]. Samples that require fewer partitions to isolate are scored as more anomalous, since they differ more from the majority of the data. OCSVM maps the training data into a high-dimensional feature space and learns a hyperplane boundary enclosing the normal distribution, flagging any test sample that falls outside this boundary. PCA-based reconstruction identifies outliers by measuring how accurately each sample can be reconstructed from the principal components of the normal training data.

While these methods have demonstrated utility in network intrusion detection and log analysis, they have a critical limitation when applied to APT detection. The behavioral indicators of an APT attack are rarely statistically extreme in isolation. An individual log event may look entirely normal when evaluated independently. The attack only becomes visible when events are examined in their sequential and contextual relationships. Classical methods evaluate each sample independently and therefore systematically miss the multi-event context that APT campaigns rely on to remain undetected [9].

This limitation has motivated a shift toward deep learning-based anomaly detection, with autoencoders emerging as the dominant approach. By learning non-linear representations of normal behavior, autoencoders can detect subtle distributional deviations that statistical methods miss. A systematic review by Mutalib et al. of explainable deep learning methods for APT detection [9] found that autoencoder-based systems consistently achieved detection accuracies between 96.7% and 98% across benchmark datasets, substantially outperforming classical baselines on multi-stage, sequential attack patterns.

2.3. Large Language Models in Cybersecurity

LLMs have been used for various Natural Language Processing (NLP) applications such as Misinformation detection [19], Debiasing media [20], Detecting harmful content [21] and political leaning [22], and medical understanding and reasoning [23]. The application of LLMs to cybersecurity tasks has expanded rapidly in recent years, driven by their ability to encode contextual semantic relationships and generalize across domains without task-specific training data. Ali et al. conducted a comprehensive survey of LLM applications across eight cybersecurity domains, including vulnerability detection, malware analysis, intrusion detection, and phishing identification [2]. Their analysis identified a fundamental advantage of LLM-based systems over rule-based alternatives: rather than matching input against fixed patterns, LLMs reason about the meaning of event sequences, enabling detection of behaviors that have no prior signature. The survey further noted that encoder-only architectures such as BERT and RoBERTa consistently outperform encoder-decoder models on classification and

anomaly detection tasks, while encoder-decoder models are more suitable for generation tasks such as automated threat report synthesis.

With a focus specifically on APT detection, Aziz et al. reviewed the applicability of models including BERT, RoBERTa, GPT-3, and LLaMA2 for automated threat intelligence extraction and log-based anomaly detection [11]. Their analysis confirmed strong potential across these models but identified two practical barriers to deployment: the significant computational overhead of large-scale models, and the difficulty of fine-tuning these models for cybersecurity tasks given the chronic scarcity of labelled APT incident data.

Jaffal et al. conducted a broader systematic review spanning 172 studies on LLM applications in cybersecurity [13]. Despite strong reported performance, the review consistently identified three unresolved challenges: limited interpretability of model decisions, vulnerability to adversarial perturbations, and the absence of standardized evaluation benchmarks for security-specific tasks. These findings directly motivate the approach taken in the present work. High detection performance is a necessary but insufficient condition for operational deployment in a security operations center. Without interpretable attribution, analysts cannot validate, act on, or trust the outputs of even the most accurate detection system.

Multi-agent LLM architectures have also been explored for improving detection accuracy. Rawat et al. proposed a modular system combining LLaMA 3.3-70B with Retrieval-Augmented Generation (RAG) and Chain-of-Thought prompting [4]. The system reported 93.6% detection accuracy with a 41.3% reduction in false positives relative to traditional baselines. However, the approach depends on supervised signal and incurs substantial computational overhead at inference time, making it poorly suited to the unsupervised, resource-constrained deployment conditions that characterize real-world SOC environments.

2.4. LLM Embeddings for System Log Analysis

The use of LLM embeddings for system log analysis proceeds from a straightforward intuition: dense vector representations produced by pre-trained language models encode contextual and semantic relationships that raw numerical feature vectors cannot capture. When a system log event is expressed as a natural language sentence and encoded by an LLM, the resulting vector reflects not only the individual event but its behavioral context, which is precisely the information needed to distinguish normal from malicious system activity in APT scenarios.

The most directly relevant prior work is the APT-LLM framework of Benabderrahmane et al. [1], which forms the Phase 1 baseline of the present study. In that work, system provenance traces from the DARPA TC dataset were converted into natural language sentences and encoded using five pre-trained models: BERT, ALBERT, RoBERTa, DistilBERT, and MiniLM, producing embeddings of 384 to

1024 dimensions. The resulting embeddings were fed into three autoencoder variants: a Baseline AE, a VAE, and a DAE. The optimal configuration, ALBERT paired with a VAE, outperformed classical baselines including OC-SVM, DBSCAN, and Isolation Forest. While this result establishes the viability of the embedding-based approach, the study evaluated only five embedding models from a single architectural family, leaving open the question of whether other model families might yield substantially different performance characteristics on APT telemetry.

Zuo et al. proposed a complementary approach in which GPT-4o is used to generate semantically enriched natural language descriptions of system events before encoding them with OpenAI's `text-embedding-3-small` model [12]. The enriched descriptions incorporate system call context, software identity, and execution semantics that raw logs do not express. This approach achieved 99.1% classification accuracy with an MLP classifier on the ProvSec dataset and additionally resolved the out-of-vocabulary (OOV) problem that causes static embedding methods such as Doc2Vec to fail on novel command-line syntax, where Doc2Vec had achieved only 45–62% accuracy on the same data. The significant performance gap between static and contextual embeddings in this comparison is one motivation for the systematic multi-family benchmarking conducted in the present work.

Taken together, these studies strongly suggest that the choice of embedding model has a substantial impact on downstream anomaly detection performance. However, no existing study has conducted a controlled comparison across the full range of embedding architectural families, including encoder-based, decoder-based, angle-optimized, and static models, on APT-specific log data. This gap is a central contribution of the present work.

2.5. Autoencoder Architectures for Anomaly Detection

Autoencoders trained exclusively on normal data have become the dominant approach for unsupervised anomaly detection in security applications. The core principle is well-established: by minimizing reconstruction error on normal inputs during training, the network develops internal representations optimized for normal patterns. At inference time, anomalous inputs, which the network has never been trained to reconstruct, produce elevated reconstruction errors that serve as continuous anomaly scores [13, 24].

The Vanilla Autoencoder provides the simplest implementation of this principle and serves as the natural baseline for architectural comparison. The Sparse Autoencoder imposes an L1 regularization penalty on the hidden layer activations, constraining the model to activate only the most informative neurons for any given input [25], which tends to produce more selective and interpretable feature representations. The Denoising Autoencoder [26] trains the model to reconstruct

clean inputs from intentionally corrupted versions, improving robustness to the noise and missing fields commonly encountered in real system telemetry.

The Variational Autoencoder (VAE) replaces the deterministic latent encoding of the Vanilla AE with a probabilistic framework, learning a distribution over latent representations rather than a fixed point [27]. Anomalies tend to map to low-probability regions of this learned distribution, making the probabilistic framework potentially more sensitive to distributional deviations than reconstruction error alone. The β -VAE introduces a weighted regularization term to encourage disentanglement of the latent dimensions, while the Gaussian Mixture VAE (GMVAE) [28] extends the framework to a mixture prior, allowing the model to represent multiple distinct modes of normal behavior. This is particularly relevant in APT datasets where benign activity is highly heterogeneous across different system processes, user sessions, and application types.

Attention-based autoencoders incorporate self-attention mechanisms to dynamically weight the relevance of different input dimensions [29], which is well-suited to log data where the informativeness of individual fields varies substantially across events. The LSTM Autoencoder models log sequences as temporal data, capturing ordering and timing dependencies between consecutive events. The value of sequence-aware processing was demonstrated by Afnan et al. in the LogShield framework [30], where a transformer-based temporal model outperformed LSTM baselines on DARPA datasets, achieving an F1-score of 98.4% on data with extreme class imbalance. Deep SVDD [31] departs from the reconstruction paradigm entirely, instead training a neural network to map all normal data to a compact hypersphere in latent space and detecting anomalies by their distance from the hypersphere center. The Robust Autoencoder applies matrix decomposition to separate normal inputs into a low-rank background component and a sparse anomalous component, providing resilience against contaminated training data. The Contractive Autoencoder adds a Jacobian-based regularization term that penalizes sensitivity of the encoder output to small input perturbations [32], improving representational stability and generalization.

Despite the breadth of this literature, no existing study has evaluated all of these architectural variants within a single controlled experimental framework on APT-specific log embeddings. Individual studies typically compare two to three variants in isolation, leaving the relative merits of the full architectural space on this specific task unknown.

2.6. Transformer-Based Models for APT Detection

Transformer architectures have also been applied directly to APT detection, treating log sequences as inputs to full self-attention models rather than using transformers solely as feature extractors. LogShield, proposed by Afnan et al. [30], converts provenance graph traces into structured log key sequences and feeds them into a RoBERTa-based transformer augmented with a custom temporal embedding layer.

The temporal embedding uses a sliding window mechanism to encode the timing patterns characteristic of APT's slow, methodical progression through a target network, a signal that standard position-based embeddings are unable to capture. On the DARPA OpTC dataset, where malicious events constitute just 0.0016% of all records, LogShield achieved an F1-score of 98.4%, outperforming both LSTM and BERT baselines.

Aly et al. proposed OCR-APT [33], which pairs a graph convolutional network for subgraph-level anomaly detection with a GPT-4o-based investigation layer that generates structured attack narratives aligned to the APT kill chain. On the DARPA TC3 and OpTC datasets, the system achieved an average F1-score of 0.96. RAG and Chain-of-Thought prompting are incorporated to mitigate hallucinations in the generative layer, addressing a known reliability challenge with LLM-based report generation in high-stakes domains. However, the computational cost of GPT-4o inference and the reliance on supervised signals significantly limit the practical scalability of this approach in real SOC deployments.

These works confirm that sequential and graph-based modeling of log data provides detection advantages over flat feature-vector approaches. They also highlight an important deployment trade-off: systems that achieve the highest performance often carry the highest computational and operational cost. The present work is positioned at a different point on this trade-off curve, prioritizing an unsupervised, embedding-based approach that is systematically evaluated across a wide configuration space and paired with interpretable attribution.

2.7. Explainability in Security AI Systems

The integration of explainability into AI-driven security systems has moved beyond a research aspiration and is increasingly recognized as a practical operational requirement. Security analysts cannot act confidently on alerts they do not understand. In high-stakes environments such as SOCs, where false positives carry real investigation costs and missed detections carry real security risk, opaque model outputs actively impede effective threat response [14]. Regulatory frameworks governing critical infrastructure sectors increasingly require that automated decisions be auditable and explainable, adding compliance pressure to the operational motivation.

SHAP (SHapley Additive exPlanations) [16] has become one of the most widely adopted post-hoc explanation methods in the security domain. Grounded in cooperative game theory, SHAP computes the marginal contribution of each input feature to a model's prediction, with theoretical guarantees of consistency and local accuracy. This produces exact numerical attribution values that tell an analyst precisely how much each feature raised or lowered the model's output. Applied to APT detection alongside XGBoost on the

Table 1

Research gaps in existing literature and corresponding contributions of the present work.

#	Gap	Limitation in Prior Work	Contribution of This Work
1	No systematic embedding model comparison on APT data	APT-LLM [1] and Zuo et al. [12] each evaluate a small fixed set of models from a single architectural family. No cross-family comparison exists on APT-specific telemetry.	17 embedding models across 5 architectural families (Encoder, Decoder, Large, Angle-Optimized, Static) are benchmarked under identical experimental conditions on DARPA TC data.
2	Limited autoencoder architecture diversity	Existing studies compare 2-3 autoencoder variants in isolation [1, 13]. Probabilistic, sequence-aware, and robustness-oriented architectures have not been evaluated together in a single controlled experiment on APT log embeddings.	11 autoencoder topologies, including VAE, GMVAE, LSTMAE, Robust AE, Deep SVDD, and Contractive AE, are benchmarked within the same experimental pipeline.
3	Explainability rarely applied to deep unsupervised detectors on LLM embeddings	SHAP has been applied effectively to supervised classifiers with hand-crafted features [9]. Its application to embedding-based autoencoder pipelines in an unsupervised setting is largely absent from the literature.	A dual XAI pipeline provides token-level SHAP attribution via TF-IDF representations and a multi-stage ablation study that empirically links token removal to measurable shifts in the anomaly score.

SCVIC-APT-2021 dataset, SHAP-based attribution successfully identified specific network flow features as primary discriminators of APT activity classes [26], demonstrating the practical value of feature-level explanation for alert triage.

A systematic review by Mutalib et al. covering 100 studies on explainable deep learning for APT detection found a consistent empirical pattern: systems incorporating explainability mechanisms produced higher analyst trust and measurably faster incident response compared to equivalent black-box alternatives [9]. Despite this evidence, the application of SHAP to deep unsupervised anomaly detectors operating on high-dimensional LLM embeddings remains largely unexplored in the literature. Existing work applies SHAP and LIME predominantly to supervised classifiers with interpretable, hand-crafted input features. When the input representation is a dense embedding produced by a transformer model, the attribution problem changes fundamentally: individual dimensions carry no human-readable meaning, and standard feature attribution cannot be directly applied. This gap, between token-level human understanding and latent-space anomaly scoring, is the central explainability challenge that the present work addresses.

2.8. Summary and Research Gaps

Table 1 summarizes the key gaps identified across the reviewed literature and maps each gap to the specific contribution of the present work.

These three gaps collectively define the scope of the present work. No existing study has addressed all three within a unified experimental framework. The systematic benchmarking of embedding models and autoencoder architectures provides the empirical foundation needed for practitioners to make principled model selection decisions in APT detection deployments. The dual XAI pipeline addresses the broader

problem of making deep unsupervised anomaly detectors operationally trustworthy, bridging the gap between high-dimensional representations and the human-readable attribution that security analysts require.

3. Research Methodology

This section describes the complete methodology used to build and evaluate the proposed anomaly detection framework. The system follows a dual-pipeline design. The first pipeline performs semantic anomaly detection by encoding system audit logs as dense vector embeddings using pre-trained LLMs and training an unsupervised Autoencoder to identify deviations from normal behavior. The second pipeline provides explainability by tracing each anomaly score back to specific tokens in the original log text, giving security analysts a clear and actionable reason for every alert. Figure 1 provides an overview of the full end-to-end architecture.

3.1. Dataset and Anti-Leakage Protocol

The framework is evaluated on the ADAPT repository (Audit Data for APT Analysis), which is built on telemetry collected by the DARPA Transparent Computing (TC) program [34]. The data was captured by five monitoring performers called 5DIR, CADETS, CLEARSCOPE, THEIA, and TRACE, which were running across BSD, Linux, Windows, and Android operating systems. Each record in the dataset represents a single system-level event, such as a process execution, file access, network connection, or registry modification, along with associated metadata including the subject entity, object entity, action type, and timestamp.

Table 2 summarizes the dataset statistics. The most notable characteristic of this data is its extreme class imbalance. Malicious events constitute between 0.005% and 0.107% of total records, depending on the scenario, which accurately reflects real-world APT environments where attackers deliberately minimize their operational footprint to avoid detection.

To prepare the data for downstream processing, all legacy uuid columns were standardized to a universal Object_ID schema. This ensures consistent entity tracking and allows processes and network connections to be accurately correlated across different log types.

A strict anti-leakage data split was enforced throughout all experiments. Ground-truth labels were used exclusively for partitioning and final evaluation, never during training. The split is defined as follows:

- **Training set:** 80% of benign records only. No anomalous events are included. This forces the Autoencoder to build a representation of normal behavior without any exposure to attack patterns.
- **Test set:** The remaining 20% of benign records combined with 100% of the anomalous records. The model is therefore evaluated against attack vectors it has

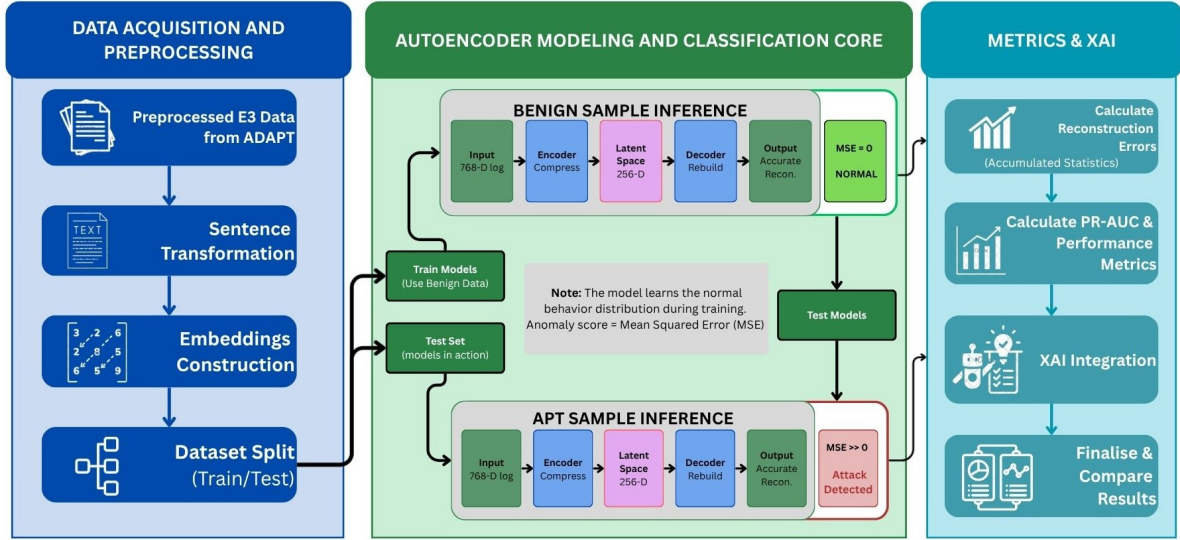


Figure 1: Overall System Architecture

Table 2

Experimental datasets of DARPA's TC program used in our study. A dataset entry (columns 3 to 7) is described by the number of rows (processes) / the number of columns (attributes). For instance, with ProcessAll (PA) obtained from the second scenario of Linux, the dataset comprises 279,369 rows and 457 attributes, including 100 APT attacks (0.036%).

	Size	PE	PX	PP	PN	PA	nb_attacks	% $\frac{nb_attacks}{nb_processes}$
BSD	265 MB	224624 / 32	224146 / 136	223780 / 38	42888 / 63	224624 / 266	11	0.005
Windows	36.0 MB	11151 / 31	11077 / 372	10992 / 81	329 / 126	11151 / 607	9	0.081
Linux (1)	3.63 GB	282087 / 26	271088 / 141	263730 / 46	6580 / 6226	282103 / 6436	46	0.016
Linux (2)	465 MB	270652 / 19	278363 / 160	279090 / 71	48048 / 210	279369 / 457	100	0.036
Android	18.1 MB	12106 / 28	12106 / 45	24 / 12	1116 / 214	12106 / 296	13	0.107

never encountered during training, which faithfully simulates real-world zero-day detection conditions.

Figure 2 visualizes the split logic used in the research. To ensure a fair evaluation, all anomalous records are completely excluded from the training phase and reserved for testing only. This protocol allows the models to learn normal behavioral patterns from benign data while accurately measuring their ability to identify previously unseen anomalous activities during inference.

3.2. Log-to-Sentence Conversion and Embedding Extraction

Raw audit logs contain structured tabular fields that pre-trained NLP models cannot directly process. Each record r_i from the provenance database $\mathcal{D} = \{r_1, r_2, \dots, r_N\}$ is therefore converted into a natural language sentence s_i that preserves both the behavioral action and the structural context. For example, a process record associated with `dismhost.exe` is mapped to:

Process has event check_file_attributes and event create_thread and executable \users\admin\appdata\local\temp\dismhost.exe.

Each sentence s_i is then passed through a pre-trained LLM tokenizer and encoder to produce a fixed-size dense embedding $x_i \in \mathbb{R}^d$. To aggregate token-level hidden states into a single sequence representation, attention-weighted mean pooling is applied:

$$x_{\text{pooled}} = \frac{\sum_{j=1}^L (h_j \cdot m_j)}{\max(\sum_{j=1}^L m_j, \epsilon)} \quad (1)$$

where h_j is the hidden state of the j -th token, m_j is its binary attention mask value (which prevents padding tokens from contributing to the mean), and ϵ is a small constant that prevents division by zero. The resulting vector is then L2-normalized to project it onto a unit hypersphere, yielding the final embedding $x_i \in \mathbb{R}^d$.

3.3. LLM Embedding Model Benchmarking

A key contribution of this study is the systematic benchmarking of 17 pre-trained embedding models across five architectural families. To the best of our knowledge, this is the first study to compare encoder-based, decoder-based, angle-optimized, and static embedding families under identical

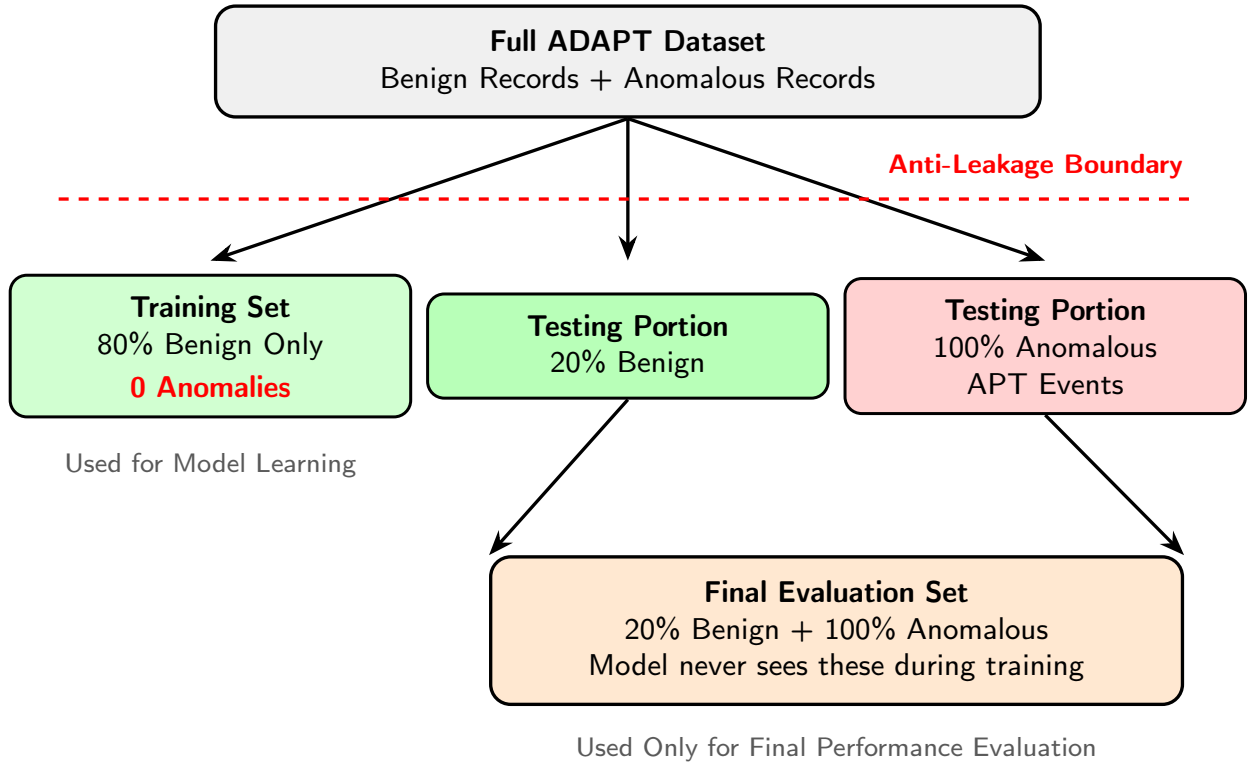


Figure 2: Improved anti-leakage 80/20 dataset split protocol. The model is trained exclusively on benign records, while all anomalous records are reserved for testing. This ensures unbiased evaluation and prevents data leakage.

controlled conditions on APT-specific log data. The models are organized as follows, and fully detailed in Table 3.

- **Group A – Encoder (BERT-based):** Bidirectional models that process each token in context of all surrounding tokens. This architecture is well suited to structured log syntax, where the relationship between a process name, its arguments, and its parent process carries meaning. Models include facebook/contriever [35], BAAI/bge-m3 [36], sentence-transformers/all-mpnet-base-v2 [37], jinaai/jina-embeddings-v3 [38], nommic-embed-text-v1.5 [39], infloat/multilingual-e5-large [40], Alibaba-NLP/gte-large-en-v1.5 [41, 42], and answerdotai/ModernBERT-large [43].
- **Group B – Decoder (LLM-based):** Left-to-right generative models adapted for embedding tasks. Their large parameter counts and emergent reasoning capabilities make them interesting candidates, though their instruction-tuning for natural language generation may not transfer to structured telemetry. Models include nvidia/NV-Embed-v2 [44, 45], Alibaba-NLP/gte-Qwen2-7B-instruct [42], infloat/e5-mistral-7b-instruct [46, 47], Salesforce/SFR-Embedding-Mistral [48], ling-ai/ling-embed-mistral [49], and McGill-NLP/LLM2Vec-Llama-3-8B-mntp [50].
- **Group C – Large-scale:** sentence-transformers/gtr-t5-large [51], a scaled encoder-decoder model.

Table 3

Taxonomy of LLM embedding models evaluated in this study, organized by architectural group.

#	Model	Group	Dim.	Key Characteristic
1	facebook/contriever [35]	A – Encoder	768	Unsupervised dense retriever
2	BAAI/bge-m3 [36]	A – Encoder	1024	Multilingual multi-task
3	gte-large-en-v1.5 [40]	A – Encoder	1024	Semantic similarity
4	all-mpnet-base-v2 [37]	A – Encoder	768	Strong sentence-level baseline
5	jina-embeddings-v3 [38]	A – Encoder	1024	Long-context (8192 tokens)
6	nommic-embed-text-v1.5 [39]	A – Encoder	768	Open-source, large context
7	ModernBERT-large [43]	A – Encoder	1024	Extended context BERT variant
8	multilingual-e5-large [40]	A – Encoder	1024	Extended XLM-RoBERTa framework
9	nvidia/NV-Embed-v2 [44, 45]	B – Decoder	4096	Instruction-tuned decoder
10	gte-Qwen2-7B-instruct [42]	B – Decoder	3584	7B instruction-tuned decoder
11	e5-mistral-7b-instruct [46, 47]	B – Decoder	4096	Mistral embedding model
12	SFR-Embedding-Mistral [48]	B – Decoder	4096	Retrieval-optimized mistral
13	ling-embed-mistral [49]	B – Decoder	4096	Instruction-tuned semantic search
14	LLM2Vec-Llama-3-8B-mntp [50]	B – Decoder	4096	LLaMA-3 bidirectional encoder
15	gtr-t5-large [51]	C – Large	768	T5 encoder-decoder
16	WhereIsAI/UAE-Large-V1 [52]	D – Angle-optimized	1024	Angle-based contrastive
17	stella_en_400M_v5 [53]	E – Static	1024	Lightweight, fixed repr.

- **Group D – Angle-optimized:** WhereIsAI/UAE-Large-V1 [52], which uses angle-based contrastive learning to prevent representation collapse at the extremes of the embedding space.
- **Group E – Static:** dunzhang/stella_en_400M_v5 [53], a lightweight fixed-representation model included as an efficiency baseline.

To analyze how these different dimensionality settings and architectural choices influence the separation of system log

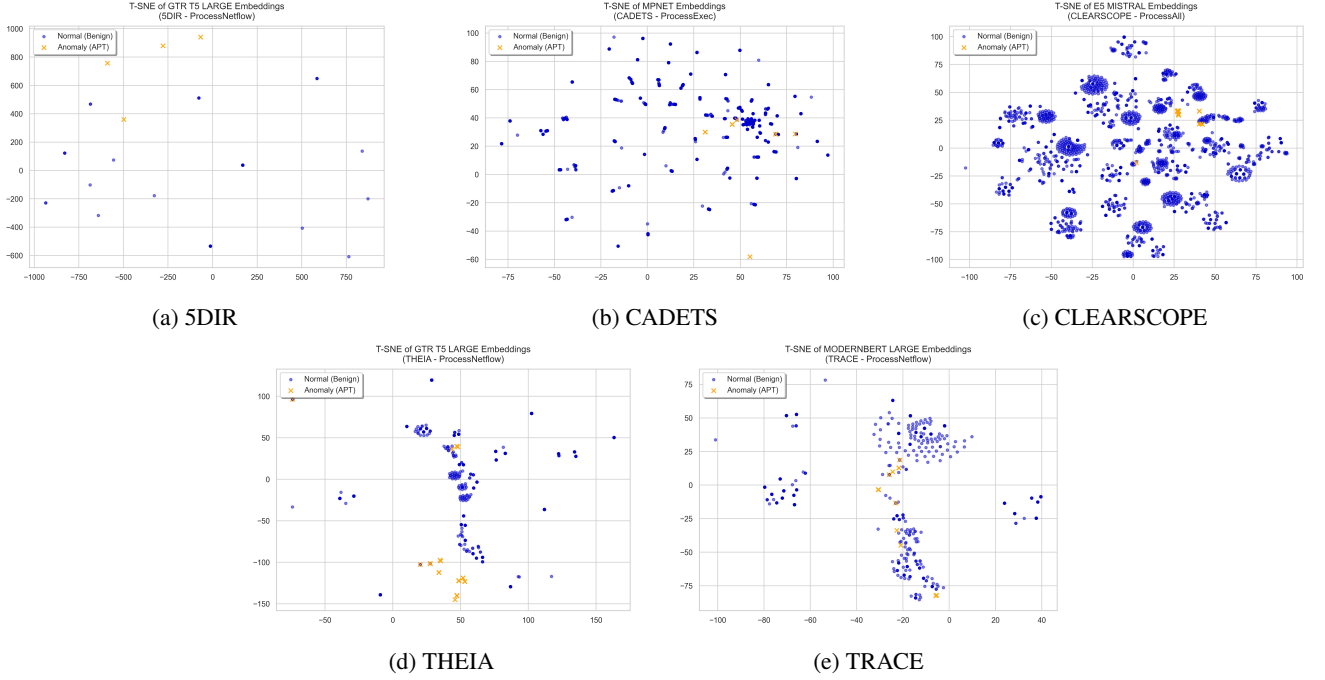


Figure 3: Two-dimensional t -SNE projection comparison of representative raw embedding groups. Malicious logs are marked in red while normal activities are marked in blue, demonstrating the initial cluster density before neural autoencoder filtering.

activities, high-dimensional raw embeddings are visualized using t -Distributed Stochastic Neighbor Embedding (t -SNE). Figure 3 shows the t -SNE projections of log vectors before they are processed by the anomaly detection pipeline. For visualization, we use the best-performing combination of the embedding model and autoencoder. This helps us clearly observe whether certain model setups naturally separate anomalous log patterns from normal (benign) sequences in the original latent space.

3.4. Autoencoder Architecture Search

The Autoencoder serves as the anomaly detection engine. It is trained exclusively on benign log embeddings and learns to and reconstruct normal behavior. When it encounters an anomalous input during inference, the reconstruction fails, and the resulting error becomes the anomaly score. Formally, given an input embedding $x \in \mathbb{R}^d$, an encoder function f_θ compresses it to a latent representation $h \in \mathbb{R}^k$ where $k < d$:

$$h = f_\theta(x) = \sigma(Wx + b) \quad (2)$$

A decoder function g_ϕ then reconstructs the original input from h :

$$\hat{x} = g_\phi(h) = \sigma'(W'h + b') \quad (3)$$

Both functions are jointly optimized to minimize the Mean Squared Error (MSE) over all benign training samples:

$$\mathcal{L}_{\text{MSE}}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|x_i - g_\phi(f_\theta(x_i))\|^2 \quad (4)$$

During inference, an APT log vector x_{apt} produces a high MSE because its patterns fall outside the normal distribution the model has learned. This MSE value directly serves as the continuous anomaly score for threshold-based classification.

To identify the best-performing architecture, 11 distinct Autoencoder topologies were implemented and evaluated. Table 4 summarizes each architecture, its mathematical regularization strategy, and whether it is deterministic or probabilistic. This classification is significant because, as discussed in Section 3.5, the explainability pipeline requires deterministic behavior to produce stable SHAP attributions.

Table 5 shows the corresponding network architectures, key hyperparameters, and optimization objectives used during training. All models were trained under identical optimization settings to ensure a fair comparison across architectures.

Probabilistic topologies (VAE, β -VAE, GMVAE) were disqualified because their stochastic sampling process introduces variability in the latent space. SHAP computes attribution values by measuring small, precise changes in model output in response to input perturbations. This requires the model to produce identical outputs for identical inputs across multiple evaluations. The non-determinism of VAE sampling violates this requirement and produces unreliable attribution values [16].

Table 4

Autoencoder topologies evaluated in the architecture search, with their regularization strategies and determinism properties.

Topology	Regularization / Objective	Deterministic
Baseline AE [25]	(Vanilla) Standard MSE reconstruction loss	Yes
Sparse AE [54]	L1 sparsity penalty on hidden activations	Yes
Denoising AE (DAE) [55]	Reconstruct clean input from corrupted version	Yes
Robust AE [56]	Low-rank + sparse matrix decomposition	Yes
Contractive AE [32]	Frobenius norm penalty on encoder Jacobian	Yes
Attention AE [57]	Self-attention weighting on input features	Yes
LSTMAE [58]	Sequential temporal modeling via LSTM layers	Yes
Deep SVDD [59]	Hypersphere boundary, no reconstruction	Yes
Variational (VAE) [60]	KL-divergence to Gaussian prior	No
β -VAE [61]	Weighted KL-divergence, disentangled latent space	No
GMVAE [62]	Gaussian Mixture prior, multi-modal latent space	No

Table 5

Autoencoder architectures and hyperparameters used during the architecture search.

Model	Base Architecture	Key Hyperparameters	Objective Function
Baseline AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	N/A	MSE
Sparse AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Sparse Penalty = 10^{-4}	MSE + L1 Penalty
Deep SVDD	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 128)	Latent Dimension = 128	Distance to Hypersphere Center
DAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Noise Factor = 0.2	MSE
VAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Latent Dimension = 256	MSE + KLD
Beta-VAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	$\beta = 4.0$	MSE + $\beta \times$ KLD
GMVAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Components = 5	MSE
Attention AE	Multihead Attention	Attention Heads = 8	MSE
LSTM AE	LSTM (Input $\rightarrow$ 256)	Hidden State Dimension = 256	MSE
Robust AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	N/A	L1 Loss (MAE)
Contractive AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	CAE $\lambda = 10^{-4}$	MSE + Jacobian Penalty

Note: All autoencoders were trained using the Adam optimizer with a learning rate of 0.001, batch size of 128, and 15 epochs.

A systematic overview of the preprocessing pipeline and the training and scoring is shown in the Algorithm 1 below.

Algorithm 1 Log-to-Sentence Formulation, Anti-Leakage Split, and Robust AE Training

Require: Raw CSV Dataset D , Vectorizer $\mathcal{V}$, Epochs E , Batch Size B

Ensure: Trained Autoencoder $\mathcal{M}$, Anomaly Scores A_{scores}

```

1: Phase 1: Sentence Formation & Anti-Leakage Split
2: Initialize empty lists  $S_{normal}$ ,  $S_{anomaly}$ 
3: for each row  $r_i$  in  $D$  do
4:   Extract label  $y_i$ 
5:   Construct sentence  $s_i$  by concatenating column headers and values
6:   Example: "Process has event [Action] and executable [Path]"
7:   if  $y_i == 0$  then
8:      $S_{normal} \leftarrow S_{normal} \cup \{s_i\}$ 
9:   else
10:     $S_{anomaly} \leftarrow S_{anomaly} \cup \{s_i\}$ 
11:   end if
12: end for
13:  $split\_idx \leftarrow \lfloor 0.8 \times \text{length}(S_{normal}) \rfloor$ 
14:  $Train_{texts} \leftarrow S_{normal}[0 \dots split\_idx]$ 
15:  $Test_{texts} \leftarrow S_{normal}[split\_idx \dots \text{end}] \cup S_{anomaly}$ 
16: Apply  $\mathcal{V}$  on  $(Train_{texts} \cup Test_{texts})$ 
17:  $X_{train} \leftarrow \mathcal{V}.\text{transform}(Train_{texts})$ 
18:  $X_{test} \leftarrow \mathcal{V}.\text{transform}(Test_{texts})$ 
19: Phase 2: Robust Autoencoder (RAE) Training
20: Initialize Encoder  $\mathcal{E}_\theta$  and Decoder  $\mathcal{D}_\phi$  (with LeakyReLU, Dropout)
21: Define Model  $\mathcal{M}(x) = \mathcal{D}_\phi(\mathcal{E}_\theta(x))$ 
22: Define Loss  $\mathcal{L}_{L1}(x, \hat{x}) = \text{MeanAbsoluteError}(x, \hat{x})$ 
23: for  $epoch = 1 \dots E$  do
24:   for each mini-batch  $b$  in  $X_{train}$  do
25:      $\hat{b} \leftarrow \mathcal{M}(b)$ 
26:      $loss \leftarrow \mathcal{L}_{L1}(b, \hat{b})$ 
27:     Backpropagate and update weights  $\theta, \phi$ 
28:   end for
29: end for
30: Phase 3: Scoring
31: Define Anomaly Score function:  $Score(x) = \text{MSE}(x, \mathcal{M}(x))$ 
32:  $A_{scores} \leftarrow Score(X_{test})$ 
33: return  $\mathcal{M}, A_{scores}$ 

```

3.5. Dual-Pipeline XAI Attribution Framework

Generating an anomaly score is necessary but not sufficient for operational deployment. A security analyst responding to an alert needs to understand which specific elements of the log triggered the detection. The XAI framework addresses this through two complementary pipelines, each operating at a different level of abstraction. A comprehensive overview of the dual pipeline approach is shown in Algorithm 2 below.

Algorithm 2 Dual-Pipeline XAI: SHAP Attribution and Latent Space Ablation

Require: Trained Model $\mathcal{M}$, Background X_{bg} , Target Log Sequence S , Vocabulary V , LLM Embedder $\mathcal{F}_{emb}$

Ensure: Top Threat Tokens T_{top} , Ablation Trajectory $\mathcal{R}$

- 1: **Phase 1: Token-Level Attribution (SHAP)**
- 2: Define Wrapper $\mathcal{W}(x) = \text{MSE}(x, \mathcal{M}(x))$
- 3: Initialize Explainer with $\mathcal{W}$ and X_{bg}
- 4: Compute SHAP Values Φ for target log vector x_{target}
- 5: Find positive impact indices: $idx_{pos} \leftarrow \text{where}(\Phi > 0)$
- 6: Sort idx_{pos} by impact magnitude in descending order
- 7: Extract top k tokens: $T_k \leftarrow \{V[idx_{pos}[0]], V[idx_{pos}[k-1]]\}$
- 8: **Phase 2: Automated Multi-Stage Ablation**
- 9: Initialize Results list $\mathcal{R} \leftarrow \emptyset$
- 10: $e_0 \leftarrow \mathcal{F}_{emb}(S)$ $\triangleright$ Baseline LLM Embedding
- 11: $\delta_0 \leftarrow \text{MSE}(e_0, \mathcal{M}(e_0))$ $\triangleright$ Baseline Error
- 12: $\mathcal{R} \leftarrow \mathcal{R} \cup \{(S, \text{"Original"}, \delta_0)\}$
- 13: $S_{removed} \leftarrow S$
- 14: **for** each token $t \in T_{top}$ **do**
- 15: $\triangleright$ Clean whole-word deletion using regular expressions
- 16: $S_{removed} \leftarrow \text{RegExRemove}(S_{removed}, \text{boundary}(t))$
- 17: $S_{removed} \leftarrow \text{CleanExtraSpaces}(S_{removed})$
- 18: $e_{removed} \leftarrow \mathcal{F}_{emb}(S_{removed})$
- 19: $\delta_k \leftarrow \text{MSE}(e_{removed}, \mathcal{M}(e_{removed}))$
- 20: $\Delta\delta \leftarrow \delta_0 - \delta_k$ $\triangleright$ Verify drop in reconstruction error
- 21: $\mathcal{R} \leftarrow \mathcal{R} \cup \{(S_{removed}, \text{"Removed " + } t, \delta_k)\}$
- 22: **end for**
- 23: **return** $T_{top}, \mathcal{R}$

3.5.1. Pipeline 1: Token-Level Attribution via TF-IDF and SHAP

The first pipeline operates on a TF-IDF representation of the log text. TF-IDF maps each unique word in the log corpus to a distinct feature dimension, which means every feature has a direct human-readable interpretation. An Autoencoder trained on the TF-IDF representations is paired with a SHAP Explainer [16], which uses a background dataset of benign logs as the reference distribution.

SHAP computes attribution values by applying cooperative game theory. For a given flagged log, it calculates the exact marginal contribution of each token to the model's MSE output. This produces a ranked list of tokens with precise numerical impact values, directly showing which words, such as a randomized GUID fragment or a transient directory path, drove the anomaly score upward.

3.5.2. Pipeline 2: LLM Latent Space Ablation Study

Applying SHAP directly to a model embedding does not produce interpretable results, because individual embedding dimensions do not correspond to any human-readable concept. To bridge the gap between the TF-IDF attribution and the LLM's internal representations, an Automated Multi-Stage Ablation Study was designed.

The process works as follows. The top threat tokens identified by Pipeline 1 are extracted programmatically. The original log text is passed through the full pipeline to record the baseline MSE score δ_0 . The identified threat tokens are then sequentially removed. After each ablation step, the modified text is re-embedded, and the new MSE score δ_k is recorded. The resulting change in reconstruction error:

$$\Delta\delta_k = \delta_0 - \delta_k \quad (5)$$

Figure 4 presents the multi-stage ablation procedure adopted in this study. Tokens identified as highly influential by TF-IDF and SHAP are iteratively removed from the input text and re-evaluated by the anomaly detection pipeline. Changes in reconstruction error after each masking operation reveal the extent to which individual tokens contribute to the anomaly score and provide an additional layer of explanation beyond feature attribution alone.

3.6. Evaluation Metrics

Model performance is quantified using three metrics selected specifically for the challenges of APT detection, where the class distribution is severely imbalanced and the cost of both missed detections and false alarms is operationally significant.

Reconstruction Error (MSE). The continuous anomaly score for each log entry. For an input vector x of dimension D and its reconstruction $\hat{x}$:

$$\text{MSE}(x, \hat{x}) = \frac{1}{D} \sum_{i=1}^D (x_i - \hat{x}_i)^2 \quad (6)$$

A higher MSE indicates the model could not accurately reconstruct the input, signaling a likely anomaly.

AUC-ROC. Evaluates the model's ability to separate anomalous from benign events across all possible classification thresholds. It is defined as the area under the curve plotting the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(t)) dt \quad (7)$$

where $\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ and $\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$. A score of 1.0 represents perfect class separation.

PR AUC. Because APT datasets are severely imbalanced, AUC-ROC alone can be misleading. A model that classifies everything as benign achieves a high AUC-ROC simply because normal events dominate the test set. PR AUC addresses this by measuring performance specifically on the minority anomaly class:

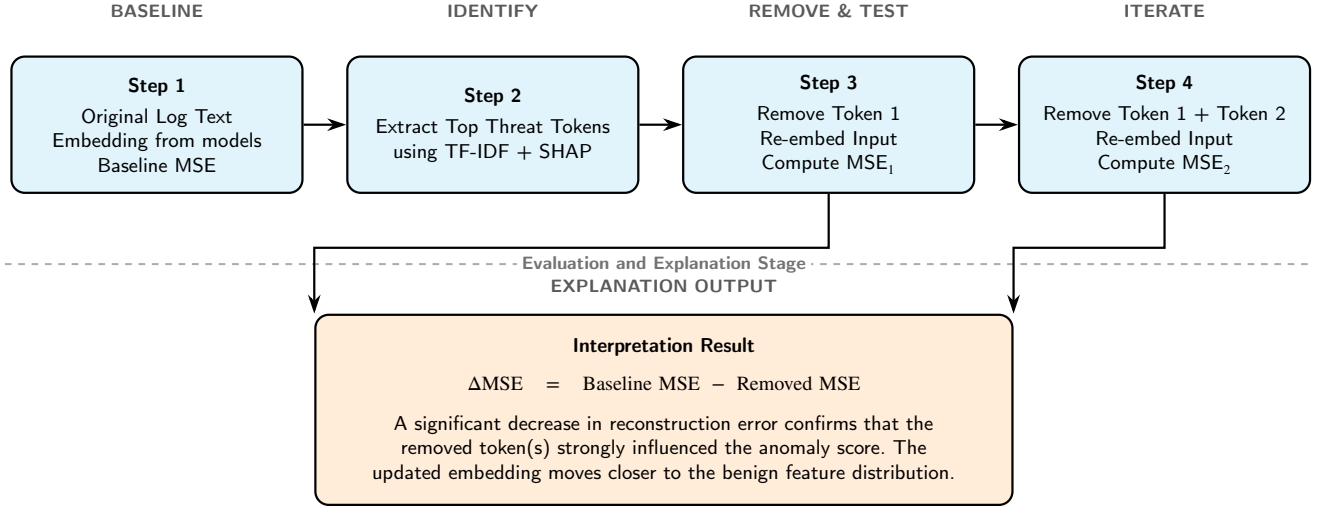


Figure 4: Enhanced automated multi-stage ablation workflow with improved alignment and flow structure. Threat-indicative tokens identified by TF-IDF SHAP are progressively removed and re-evaluated. The resulting reduction in reconstruction error quantitatively reveals which tokens most strongly drive the anomaly signal.

$$\text{PR-AUC} = \int_0^1 P(R) dR \quad (8)$$

where Precision $P = \frac{TP}{TP+FP}$ and Recall $R = \frac{TP}{TP+FN}$. A high PR AUC indicates the model identifies real anomalies without generating excessive false alarms, which is the operationally relevant measure for SOC deployment.

3.7. Implementation Details

All experiments were implemented in Python using PyTorch [63], transformers [64], shap [16], and scikit-learn [65]. GPU acceleration was provided by two NVIDIA GPUs: an RTX 1080 Ti (11 GB VRAM) and an RTX 5060 Ti (16 GB VRAM) running CUDA. Embedding generation for the larger decoder-based models (Group B, up to 4096 dimensions) was batched to fit within available VRAM.

4. Results and Discussions

4.1. Experimental Overview

This section presents all the experimental results from the full benchmarking study and explains their meaning. The evaluation tests 17 pre-trained LLM embedding models paired with 11 autoencoder architectures. This creates 187 unique combinations for each context and performer. The evaluation uses five DARPA TC performers (5DIR, CADETS, CLEARSCOPE, THEIA, and TRACE) and five process contexts (ProcessAll, ProcessEvent, ProcessExec, ProcessNetflow, and ProcessParent).

The dataset has a severe class imbalance. APT events make up as little as 0.005% of the total records in some performers. Therefore, PR-AUC is the main evaluation metric used in this section. AUC-ROC rewards a model for correctly ranking normal events above anomalous ones across all

thresholds. When a dataset has 99.995% normal events, a model can get a high AUC-ROC score just by classifying the majority class well. PR-AUC avoids this issue. It directly measures how precisely the model finds the rare positive APT class and how many of those rare events it actually catches. A model that creates many false alarms will have low precision and a low PR-AUC score. For a real Security Operations Center (SOC) environment where false alarm fatigue is a major problem, PR-AUC is the most honest metric.

All results were produced using the strict anti-leakage protocol described in the methodology. Ground truth labels were never used during training. The models only saw benign data during training. They were evaluated on a test set that contained the held-out 20% of benign samples mixed with 100% of the APT events, which the models had never seen before.

4.2. ProcessAll Context: Best Combinations Per Performer

The ProcessAll context combines all process-level event types into a single log format. It captures process executions, file accesses, network connections, and registry modifications together. It is the broadest context in the evaluation and serves as the starting point for the analysis. Table 6 shows the best embedding and autoencoder combination for each performer under this context.

The results show a clear split. CLEARSCOPE, THEIA, and TRACE all achieve a PR-AUC above 0.83. The 5DIR and CADETS performers score much lower at 0.39 and 0.48. To understand this gap, we must look at the dataset data instead of the models.

The 5DIR performer runs on BSD and only contains 11 malicious events in the entire dataset. Measuring PR-AUC on a positive class of 11 samples is very unstable. Missing

Table 6

Best embedding and autoencoder combination per performer under the ProcessAll context, ranked by PR-AUC.

Performer	Best Combination	PR-AUC
5DIR	gte-Qwen2-7B-instruct + Contractive AE	0.3939
CADETS	BAAI/bge-m3 + LSTMAE	0.4839
CLEARSCOPE	SFR-Embedding-Mistral + Deep SVDD	0.8682
THEIA	multilingual-e5-large + Baseline AE	0.8309
TRACE	gtr-t5-large + Baseline AE	0.8377

two detections drops the recall by nearly 20 percentage points. Adding two false positives changes the precision heavily. The low scores for 5DIR do not mean the framework fails. They show that evaluating anomaly detection on only 11 positive samples without labeled training data produces results with high variance. This is a statistical limitation of the dataset, not a flaw in the detection method. The CADETS performer also has a very low APT count and faces the exact same problem.

For CLEARSCOPE, THEIA, and TRACE, the results are very strong. CLEARSCOPE achieves a PR-AUC of 0.87, and both THEIA and TRACE score above 0.83. These scores were achieved using fully unsupervised training on data with less than 0.1% APT prevalence. This indicates that the embedding and autoencoder pipeline extracts a reliable signal from the system logs.

Table 6 also shows that no single model combination works best for all five performers. CLEARSCOPE works best with a Group B decoder model (SFR-Embedding-Mistral) and Deep SVDD. THEIA and TRACE work best with encoder models (multilingual-e5-large and gtr-t5-large) and a Vanilla Baseline AE. 5DIR performs best with a decoder model (gte-Qwen2-7B-instruct) and a Contractive AE. CADETS gets the best results with BAAI/bge-m3 and an LSTMAE. This variety is not random noise. It reflects real differences in log formats, operating systems, and attack behaviors across the different performers. The best model setup depends on the deployment environment. Skipping systematic benchmarking will lead to lower detection performance.

Figure 5 displays the complete PR-AUC heatmaps for all five performers evaluated under their respective best-performing contexts. Each cell represents one of the 187 unique embedding and autoencoder model combinations. Shading intensity correlates with performance, where darker cells denote a higher PR-AUC score.

The heatmaps reveal details that the summary table misses. The most important pattern is found in the CLEARSCOPE, THEIA, and TRACE heatmaps. Entire rows are consistently darker regardless of the autoencoder paired with them. This means that once a strong embedding model is selected, most autoencoder architectures perform well. The embedding model is the main bottleneck. For practical deployment, engineers should focus their optimization time on choosing

Table 7

Best PR-AUC per process context and performer across all combinations. Bold values show the best context per performer.

Performer	ProcessAll	ProcessEvent	ProcessExec	ProcessNetflow	ProcessParent
5DIR	0.3532	0.0491	0.4053	0.8304	0.0743
CADETS	0.5206	0.0057	0.6768	0.6457	0.5004
CLEARSCOPE	0.8759	0.6928	0.7716	0.7956	0.0000
THEIA	0.8527	0.8801	0.8659	0.9684	0.5865
TRACE	0.8403	0.3013	0.8359	0.8541	0.3885

the right embedding model rather than tuning the autoencoder architecture.

The 5DIR and CADETS heatmaps look entirely different. The cells are noisy and lack clear dominant rows. This is caused by the evaluation instability on very small positive classes. The grid search cannot reliably find a dominant configuration when there are so few attacks to measure.

4.3. Cross-Context Performance Analysis

Table 7 shows the highest PR-AUC found across every context and performer combination. Each value is the best PR-AUC among all combinations for that specific setup.

This table highlights several important patterns that have clear technical explanations.

ProcessNetflow is the strongest context overall. It gives the best results for three out of five performers: 5DIR (0.8304), THEIA (0.9684), and TRACE (0.8541). It is also competitive for CADETS (0.6457). The 5DIR result is very notable. In ProcessAll, 5DIR scores only 0.35. In ProcessEvent, it drops to 0.049. However, in ProcessNetflow, it jumps to 0.83. We are evaluating the exact same dataset and the exact same 11 APT events, but the detection outcome changes completely depending on the log type used.

ProcessNetflow works well because of how APTs behave. Command-and-control communication and data exfiltration require external network connections. These connections look very different from standard normal traffic. The IP addresses belong to outside ranges. The connection ports are often non-standard. These patterns do not match the normal baseline the autoencoder learned during training. When these network events are converted into text sentences, the external IPs and unusual ports become rare tokens. The autoencoder cannot reconstruct these rare tokens well and produces a high error, flagging them as anomalies. The signal is clean and reliable.

ProcessEvent and ProcessParent are the weakest contexts. ProcessEvent gets a near-zero PR-AUC for 5DIR (0.049) and CADETS (0.006). ProcessParent fails completely for CLEARSCOPE with a PR-AUC of 0.000. These contexts create a large volume of different events where APT activity looks exactly like normal system operations. For instance, a system administrator running a diagnostic script creates process events that are structurally similar to an APT moving laterally across a network. Parent-child

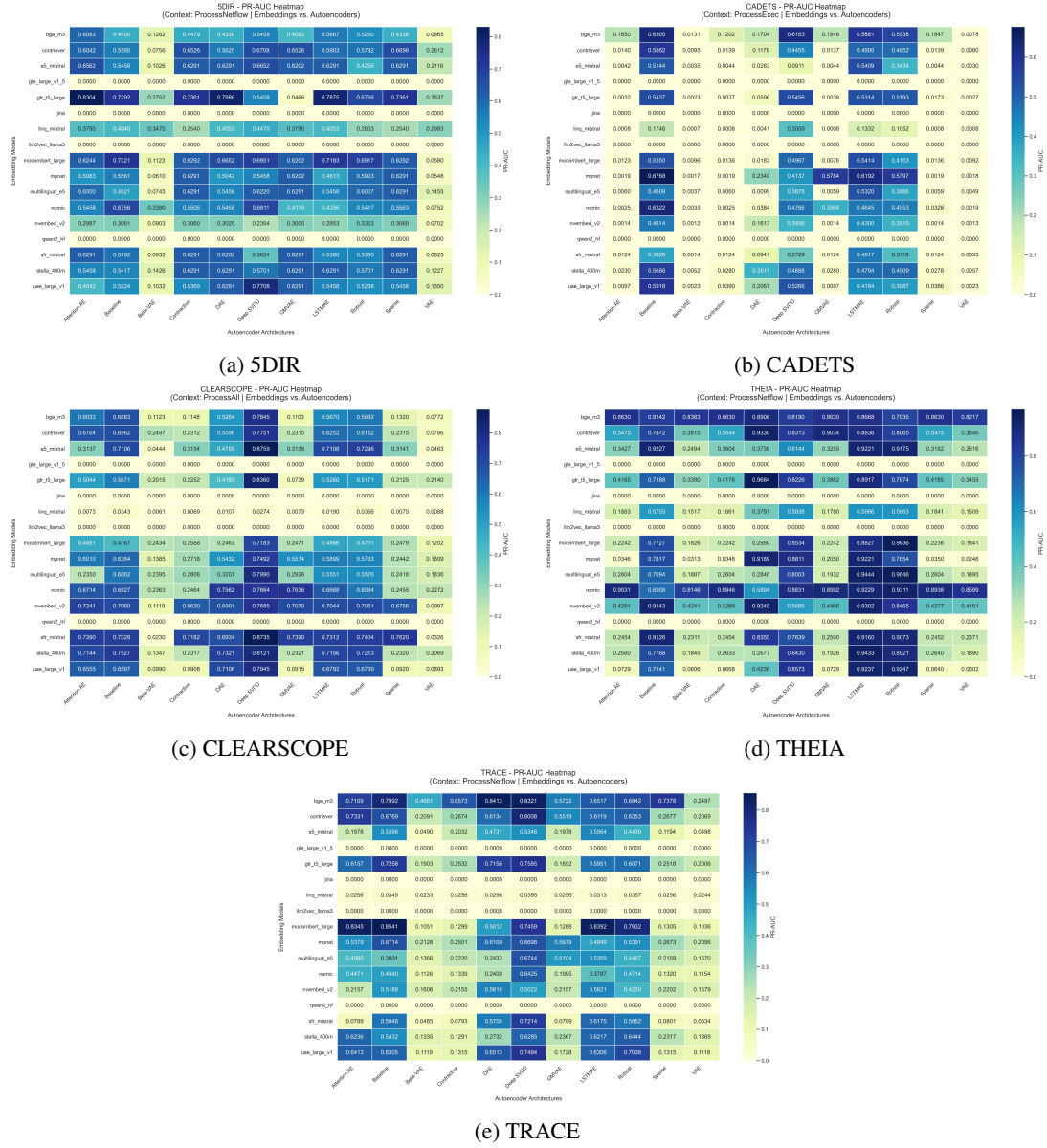


Figure 5: PR-AUC heatmaps for all five DARPA TC performers under their respective best-performing contexts. Rows represent embedding models and columns represent autoencoder architectures. Darker shading indicates higher PR-AUC.

process chains happen constantly in normal operations. The autoencoder learns to reconstruct all of this normal variety easily, which leaves no reconstruction error signal to detect the APT events.

THEIA is the most detectable performer. Its lowest PR-AUC is 0.59 in ProcessParent, and all other contexts score above 0.85. This is due to a good mix of factors. THEIA has enough APT events (46) for stable testing. It uses a structured Linux log format that maps well to the text generation template. Finally, the attacks involve distinct network and execution patterns that look clearly different from the normal training data.

The main takeaway from Table 7 is clear. Whenever network flow telemetry is available, ProcessNetflow should always be

the primary detection context. ProcessAll and ProcessExec are good backup options. ProcessEvent and ProcessParent should only be used as supplementary signals and never as the main detection input.

4.4. Best Overall Configuration Per Performer

Table 8 lists the single best configuration for each performer across all contexts and models. It includes the complete set of evaluation metrics.

THEIA with gtr-t5-large and a Denoising Autoencoder in ProcessNetflow is the strongest result. It achieves a PR-AUC of 0.9684 and a recall of 0.98, meaning it catches 98 out of 100 APT events. The specificity of 0.9423 means 94.23% of benign events are correctly classified. Obtaining this performance with unsupervised training in a dataset with

Table 8

Best overall configuration per performer with full evaluation metrics.

Performer	Best Context	Best Combination	PR-AUC	Recall	Specificity	Weighted F1
5DIR	ProcessNetflow	gtr-t5-large + Attention AE	0.8304	0.75	0.7500	0.7500
CADETS	ProcessExec	all-mpnet-base-v2 + Baseline AE	0.6768	0.5455	1.0000	0.7059
CLEARSCOPE	ProcessAll	e5-mistral-7b + Deep SVDD	0.8759	0.6923	1.0000	0.8182
THEIA	ProcessNetflow	gtr-t5-large + DAE	0.9684	0.9800	0.9423	0.9608
TRACE	ProcessNetflow	ModernBERT-large + Baseline AE	0.8541	0.7778	0.9333	0.8485

only 0.016% prevalence of APT shows that the framework works exceptionally well when correctly configured.

CADETS and CLEARSCOPE achieve perfect specificity of 1.0. At their best thresholds, these models create zero false positives. For CLEARSCOPE, Deep SVDD achieves perfect specificity with a recall of 0.6923. The model catches about 70% of APT events without a single false alarm. For CADETS, perfect specificity drops the recall to 0.5455, meaning it catches just over half the attacks. However, the PR-AUC of 0.6768 indicates there are other threshold settings available that offer higher recall if needed.

TRACE shows a highly balanced result. A PR-AUC of 0.8541 with a recall of 0.7778 and a specificity of 0.9333 shows the ModernBERT-large and Baseline AE combination catches most APTs while keeping false alarms very low.

For 5DIR, the weighted F1 of 0.75 shows the difficulty of evaluating 11 events. Both recall and specificity are at 0.75. The model detects three-quarters of the attacks and correctly classifies three-quarters of normal events. Since no attack data was used in training, this remains a good result.

Why simple autoencoders perform best. Three of the top five setups use either a Vanilla Baseline Autoencoder or a Denoising Autoencoder. Probabilistic models like VAE or GMVAE do not appear in the best results at all. The Vanilla AE works well because it learns a fixed rule for normal data. Hard anomalies, like new external IP addresses or strange file paths, easily trigger a high MSE error. The decision boundary is clean.

The Denoising AE wins for THEIA because training on corrupted inputs forces it to learn robust patterns instead of memorizing exact data. This builds a tighter normal model that highlights APT network events better. Deep SVDD wins for CLEARSCOPE because the normal data is compact enough in the embedding space to be bounded by a single hypersphere, leading to zero false positives. However, VAE models use stochastic sampling which blurs the boundary between normal and anomalous data. This can lead to missed detections. Furthermore, VAE models do not work reliably with SHAP attribution.

Encoder models outperform larger decoder models. Looking at the top setups, smaller encoder models (gtr-t5-large, all-mpnet-base-v2, ModernBERT-large) win in four out of five cases over massive 7B or 8B parameter decoder models.

Decoder models are built for general language generation. System logs are structured, repetitive templates with fixed fields. Contrastive encoder models are better at placing structurally different sentences far apart in the embedding space. This geometric separation is exactly what an autoencoder needs to find anomalies.

4.5. XAI Pipeline 1: SHAP Token Attribution Results

The first XAI pipeline provides token-level explanations. For each detected APT event, SHAP computes an exact numerical value for every word token in the log. This creates a ranked list showing exactly which words, IPs, or identifiers caused the anomaly score to rise.

CLEARSCOPE. In the first flagged log, the top SHAP values belong to IP octets 70, 157, and 118 from the address 53.157.70.118, associated with a `com.metasploit.stage` attribute. The model flags this because it never saw this external IP in normal traffic.

5DIR. In 5DIR, the top contributors are the IP octet 10, `ping`, and `cmd`. This is a direct signature of command-line network reconnaissance inside a Windows environment.

TRACE. For TRACE, the external address 146.153.68.151 gets high SHAP values, showing C2 beaconing. Another log shows high values for path fragments like `native`, `messaging`, and `pass_mgr`, pointing to unusual password manager access in Firefox.

CADETS. The CADETS results are dominated by the `xim` token (X Input Method server) making unusual network connections. Another log flags an `nginx` web server initiating outbound connections, which is a behavioral inversion since web servers usually only receive connections.

THEIA. THEIA logs show strong signals for C2 IP addresses on port 80. The highest single token impact belongs to `clean`, which refers to a suspicious script `/home/admin/clean.1` making outbound staging connections.

Overall, SHAP shows the model catches APTs for the right reasons. High anomaly scores are driven by external network IPs, unusual executable paths, and novel tokens that never appeared in normal operations.

4.6. XAI Pipeline 2: Latent Space Ablation Study Results

The ablation study tests a harder concept than SHAP. It evaluates whether removing the suspicious tokens results in a reduction in reconstruction error. If the error drops, it indicates that those specific words caused the high anomaly score in the dimensional latent space. Table 9 shows the ablation results.

Every single row in the table shows a drop in reconstruction error after the tokens are removed. Without exception, removing the SHAP-identified tokens lowers the error. This is true across all performers and all tested token types. The second THEIA log drops its error from 0.0016 to 0.0002

Table 9

Automated multi-stage ablation study results. The Original rows show the full log sentence and baseline error. Removed rows show which tokens were removed and the new error score. A drop in error supports attribution.

Performer	Type	Log / Removed Tokens	Mean SHAP	Recon. Error
TRACE	Original	Process has event close and event connect ... executable /etc/firefox/native-messaging-hosts/pass_mgr ... network 61.130.69.232 and network 80	3.60e-06	0.0015
	Removed	messaging, pass_mgr, hosts, native	2.59e-06	0.0011
	Original	Process has event clone ... network 146.153.68.151 and network 80 and attribute gtcache	3.66e-06	0.0015
	Removed	146.153.68.151, 80, None	1.135e-06	0.0005
THEIA	Original	Process has event clone ... executable /usr/bin/firefox and network 146.153.68.151 and network 80	1.902e-06	0.0008
	Removed	146.153.68.151, 80	6.242e-07	0.0003
	Original	Process has event clone ... executable /home/admin/profile and executable /home/admin/clean.1 ... network 161.116.88.72 and network 80	3.731e-06	0.0016
	Removed	home, admin, clean, 161, 116, 72, 80	3.535e-07	0.0002
CADETS	Original	Process has event accept ... network 10.0.4.1 and network 111 ... attribute XIM and attribute test and attribute sshd.1	5.897e-06	0.0016
	Removed	XIM, test, 111, 55, 12	4.07e-06	0.0011
	Original	Process has event accept ... network 155.162.39.48 and network 76.56.184.25 and network 80 and network 9000 and attribute nginx	7.605e-06	0.0021
	Removed	nginx, 76, 184	6.233e-06	0.0017
CLEARSCOPE	Original	Process has event bind ... network 80 and attribute com.metasploit.stage and attribute ::ffff:53.157.70.118	5.632e-06	0.0017
	Removed	157, 70, 118	1.349e-06	0.0006
	Original	Process has event bind ... network 111.82.111.27 ... network 213.186.164.125 ... attribute org.mozilla.fennec_firefox_dev	7.704e-06	0.0023
	Removed	125, 138, 213, 164, 234, 186	6.684e-06	0.0020
5DIR	Original	Process has event check_file_attributes ... executable cmd.exe.1 and attribute driverquery	2.25e-07	0.0004
	Removed	cmd, attribute	3.593e-08	0.0002
	Original	Process has event check_file_attributes ... executable \windows\system32\cmd.exe.1 and attribute ping 128.55.12.10	8.368e-07	0.0008
	Removed	10, cmd, attribute, ping	5.691e-07	0.0006

after removing just seven tokens. This is an 87.5% reduction. The CLEARSCOPE Metasploit log shows a 64.7% drop from just removing three IP octets.

These results suggest that the tokens found by SHAP are the actual cause of the anomaly detection. When an analyst sees an alert, they can trust that the highlighted tokens directly caused the alarm and represent true suspicious behavior in the log.

4.7. Summary of Key Findings

The results highlight six main conclusions:

1. **Process context is highly important.** Choosing the right log type is critical. "ProcessNetflow" provides the clearest signal and should be the primary choice.
2. **No single model fits all.** The best combination depends on the specific operating system and log format. Benchmarking is necessary for real deployments.
3. **Encoder models beat decoder models.** Smaller encoder models create better geometric separation for structured log data than massive 7B language decoder models.

4. **Simple autoencoders win.** Vanilla AE and Denoising AE provide cleaner decision boundaries and better XAI compatibility than complex probabilistic models.
5. **SHAP explanations are logical.** The framework flags external IPs, unusual paths, and novel tokens, which aligns perfectly with how human security analysts track APTs.
6. **Ablation provides evidence.** Removing suspicious tokens consistently drops the anomaly error, supporting the hypothesis that the model detects genuine adversarial data.

5. Conclusion and Future Directions

This section presents the overall conclusion and the future research directions that can help researchers contribute to this field.

5.1. Conclusion

This study introduced a fully unsupervised anomaly detection framework designed to find Advanced Persistent Threats inside massive volumes of system telemetry. By combining the textual understanding of LLMs with the anomaly-scoring power of autoencoders, we created a system that does not rely on labeled attack data.

Our extensive tests across five DARPA TC environments, 17 LLM-based embedding models, and 11 autoencoder architectures paved the way to several clear rules for real-world deployment. First, the type of log data matters most. "Network flow" logs provide the strongest and most reliable detection signal. Second, smaller encoder models consistently outperform massive language decoder models when processing structured system logs. Finally, simple autoencoders like the Vanilla or Denoising architectures are more effective and easier to explain than complex probabilistic models.

The most significant contribution of this work is the two-stage Explainable AI pipeline. By pairing SHAP token attribution with a latent space ablation study, we demonstrated that our framework detects real adversarial behavior rather than statistical noise. Security analysts can trust the system because it provides empirical proof that specific suspicious words, file paths, and external IP addresses directly caused the alert. This framework provides Security Operations Centers with a highly accurate, transparent, and practical tool to identify rare cyber threats.

5.2. Future Directions

While the current framework performs very well, several areas remain open for future research and improvement.

First, this study evaluated the models using offline batch processing. Future work must adapt the pipeline for real-time streaming environments. We need to measure the latency and throughput of generating LLM embeddings and scoring them under live, high-speed network loads to ensure the system can operate at scale in a live production environment.

Second, computer systems change naturally over time as administrators update software and configure new services. Anomaly detection models often suffer from concept drift when this new normal behavior is flagged as suspicious. Future research should explore continual learning methods. These methods would allow the autoencoder to safely update its definition of normal behavior over time without requiring a complete retrain from scratch.

Third, while we found that network flow logs are the best single source of information, attackers also leave vital traces in file accesses, memory injections, and registry changes. We plan to investigate advanced data fusion techniques. The goal is to combine multiple process contexts into a single, unified detection score without burying the strong network signal underneath noisy operating system events.

Finally, this framework was evaluated on host-based operating system logs from the DARPA TC dataset. Future work should test the pipeline on modern cloud infrastructure logs, such as Kubernetes audit logs or AWS CloudTrail telemetry. Testing in these environments will confirm that the text-based embedding approach remains highly effective across modern, cloud-native attack scenarios.

A. Appendices

This appendix presents extended experimental results, including confusion matrices, PR-AUC heatmaps, and SHAP-based interpretability analyses to support further and validate the findings discussed in the main text.

A.1. Confusion Matrices for Best Context per Performer

This section highlights the best-performing configuration for each dataset across all evaluated contexts. The selected configurations represent the optimal trade-off between precision and recall based on overall PR-AUC and confusion matrix analysis.

As shown in Figure 6, the optimal settings vary across datasets, indicating that no single configuration dominates all environments. For example, some datasets benefit more from transformer-based embeddings, while others perform better with hybrid architectures. This confirms the importance of dataset-specific tuning in anomaly detection systems.

A.2. SHAP Token Attribution Results

This section presents local interpretability results using SHAP token attributions. These visualizations explain how individual tokens in system logs influence the final anomaly score prediction.

As shown in Figures 7, 8, certain tokens consistently contribute positively toward anomaly detection, indicating their

strong correlation with abnormal system behavior. In contrast, neutral tokens have minimal impact on model predictions. Across datasets, similar token patterns emerge, suggesting that the model learns transferable semantic structures in log sequences.

A.3. SHAP Beeswarm Plots

This section provides a global explanation of model behavior using SHAP beeswarm plots. Unlike local explanations, these plots summarize feature importance across the entire dataset.

From Figure 9, it is evident that only a subset of tokens consistently contribute to high anomaly scores, while most features exhibit low or symmetric influence. The wide dispersion of points for certain tokens indicates high variability in their contribution depending on context, which reflects the dynamic nature of system logs in real-world environments.

Overall, these extended results further validate the robustness, interpretability, and generalizability of the proposed anomaly detection framework across multiple DARPA TC environments.

CRedit authorship contribution statement

Waleed Khan Mohammed: Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization, Visualization. **Hezerul Abdul Karim:** Writing – review & editing, Supervision, Project Administration, Funding Acquisition. **Vik Tor Goh:** Supervision, Formal analysis. **Nouar AlDahoul:** Writing – review & editing, Conceptualization, Methodology, Supervision, Project Administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

The publication of this article was funded by Multimedia University, Malaysia.

Data availability

The data used in this study are publicly available from the DARPA Transparent Computing (TC) dataset repositories. All datasets are openly accessible through their official distribution channels. <https://github.com/darpa-i2o/Transparent-Computing>

References

- [1] S. Benabderrahmane, P. Valtchev, J. Cheney, and T. Rahwan, "Apt-llm: embedding-based anomaly detection of cyber advanced persistent threats using large language models," in *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*, pp. 1–6, IEEE, 2025.
- [2] A. Ali and M. C. Ghanem, "Beyond detection: Large language models and next-generation cybersecurity," *SHIFRA*, vol. 2025, p. 81–97, Feb. 2025.
- [3] Kaspersky, "Advanced persistent threats target one in four companies in 2024," <https://www.kaspersky.com/about/press-releases/advanced-persistent-threats-target-one-in-four-companies-in-2024>, Feb. 2025.
- [4] Y. Hmimou, M. Tabaa, A. Khiat, and Z. Hidila, "A multi-agent system for cybersecurity threat detection and correlation using large language models," *IEEE Access*, 2025.
- [5] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth IEEE international conference on data mining*, pp. 413–422, IEEE, 2008.
- [7] B. Schölkopf, R. C. Williamson, A. Smola, J. Shawe-Taylor, and J. Platt, "Support vector method for novelty detection," in *Advances in Neural Information Processing Systems* (S.olla, T. Leen, and K. Müller, eds.), vol. 12, MIT Press, 1999.
- [8] H. Abdi and L. J. Williams, "Principal component analysis," *Wiley interdisciplinary reviews: computational statistics*, vol. 2, no. 4, pp. 433–459, 2010.
- [9] N. H. A. Mutalib, A. Q. M. Sabri, A. W. A. Wahab, E. R. M. F. Abdullah, and N. AlDahoul, "Explainable deep learning approach for advanced persistent threats (apts) detection in cybersecurity: A review," *Artificial Intelligence Review*, vol. 57, no. 11, p. 297, 2024.
- [10] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.
- [11] Z. Aziz and A. Abed, "Harnessing large language models for enhanced cybersecurity: A review of their role in defending against apt and cyber attacks," *International Journal of Mechatronics, Robotics, and Artificial Intelligence*, vol. 1, no. 1, pp. 54–62, 2025.
- [12] F. Zuo, J. Rhee, and Y. R. Choe, "Knowledge transfer from llms to provenance analysis: A semantic-augmented method for apt detection," *arXiv preprint arXiv:2503.18316*, 2025.
- [13] N. O. Jaffal, M. Alkhanafseh, and D. Mohaisen, "Large language models in cybersecurity: A survey of applications, vulnerabilities, and defense techniques," *AI*, vol. 6, no. 9, p. 216, 2025.
- [14] H. S. Eriksson and G. Grov, "Towards xai in the soc – a user centric study of explainable alerts with shap and lime," in *2022 IEEE International Conference on Big Data (Big Data)*, pp. 2595–2600, 2022.
- [15] N. H. Abdul Mutalib, A. Q. M. Sabri, A. W. Abdul Wahab, E. R. Mohd Faizal, and N. Aldahoul, "A stability-aware feature selection (safs) framework for advanced persistent threats (apts) detection," *Available at SSRN 6828213*.
- [16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.
- [17] K. Sparck Jones, *A statistical interpretation of term specificity and its application in retrieval*, p. 132–142. GBR: Taylor Graham Publishing, 1988.
- [18] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [19] N. Aldahoul and Y. Zaki, "Toward a safer web: multilingual multi-agent llms for mitigating adversarial misinformation attacks," *arXiv preprint arXiv:2510.08605*, 2025.
- [20] C. W. Kuo, K. Chu, N. AlDahoul, H. Ibrahim, T. Rahwan, and Y. Zaki, "Neutralizing the narrative: Ai-powered debiasing of online news articles," *arXiv preprint arXiv:2504.03520*, 2025.
- [21] N. AlDahoul, M. J. Tan, H. R. Kasireddy, and Y. Zaki, "Guardians of digital safety: benchmarking large language models in the fight against online toxicity," *Journal of Big Data*, vol. 13, no. 1, p. 6, 2026.

- [22] N. AlDahoul, T. Rahwan, and Y. Zaki, "Polytc: a novel bert-based classifier to detect political leaning of youtube videos based on their titles," *Journal of Big Data*, vol. 11, no. 1, p. 80, 2024.
- [23] N. AlDahoul and Y. Zaki, "Nyuaad at arahealthqa shared task: Benchmarking the medical understanding and reasoning of large language models in arabic healthcare tasks," in *Proceedings of The Third Arabic Natural Language Processing Conference: Shared Tasks*, pp. 119–125, 2025.
- [24] W. K. Mohammed, Z. A. I. B. S. Anuar, M. S. M. Mitani, H. A. Karim, and N. AlDahoul, "Semantic-aware advanced persistent threat detection using autoencoders on llm-encoded system logs," 2026.
- [25] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [26] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proceedings of the 25th international conference on Machine learning*, pp. 1096–1103, 2008.
- [27] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [28] N. Dilokthanakul, P. A. Mediano, M. Garnelo, M. C. Lee, H. Salimbeni, K. Arulkumaran, and M. Shanahan, "Deep unsupervised clustering with gaussian mixture variational autoencoders," *arXiv preprint arXiv:1611.02648*, 2016.
- [29] S. A. Najafi, M. H. Asemani, and P. Setoodeh, "Attention and auto-encoder hybrid model for unsupervised online anomaly detection," *arXiv preprint arXiv:2401.03322*, 2024.
- [30] S. Afnan, M. Sadia, S. Iqbal, and A. Iqbal, "Logshield: a transformer-based apt detection system leveraging self-attention," *arXiv preprint arXiv:2311.05733*, 2023.
- [31] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in *International conference on machine learning*, pp. 4393–4402, PMLR, 2018.
- [32] S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio, "Contractive auto-encoders: Explicit invariance during feature extraction," in *Proceedings of the 28th international conference on international conference on machine learning*, pp. 833–840, 2011.
- [33] A. Aly, E. Mansour, and A. Youssef, "Ocr-apt: Reconstructing apt stories from audit logs using subgraph anomaly detection and llms," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pp. 261–275, 2025.
- [34] DARPA Transparent Computing Program, "Transparent computing engagement datasets." <https://github.com/darpa-i2o/Transparent-Computing>, 2018.
- [35] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *arXiv preprint arXiv:2112.09118*, 2021.
- [36] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," 2024.
- [37] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "Mpnnet: Masked and permuted pre-training for language understanding," *Advances in neural information processing systems*, vol. 33, pp. 16857–16867, 2020.
- [38] S. Sturua, I. Mohr, M. K. Akram, M. Günther, B. Wang, M. Krimmel, F. Wang, G. Mastrapas, A. Koukounas, A. Koukounas, N. Wang, and H. Xiao, "jina-embeddings-v3: Multilingual embeddings with task lora," 2024.
- [39] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," 2024.
- [40] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual e5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.
- [41] X. Zhang, Y. Zhang, D. Long, W. Xie, Z. Dai, J. Tang, H. Lin, B. Yang, P. Xie, F. Huang, *et al.*, "mgte: Generalized long-context text representation and reranking models for multilingual text retrieval," *arXiv preprint arXiv:2407.19669*, 2024.
- [42] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, "Towards general text embeddings with multi-stage contrastive learning," *arXiv preprint arXiv:2308.03281*, 2023.
- [43] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," 2024.
- [44] G. d. S. P. Moreira, R. Osmulski, M. Xu, R. Ak, B. Schifferer, and E. Oldridge, "Nv-retriever: Improving text embedding models with effective hard-negative mining," *arXiv preprint arXiv:2407.15831*, 2024.
- [45] C. Lee, R. Roy, M. Xu, J. Raiman, M. Shoenybi, B. Catanzaro, and W. Ping, "Nv-embed: Improved techniques for training llms as generalist embedding models," *arXiv preprint arXiv:2405.17428*, 2024.
- [46] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Improving text embeddings with large language models," *arXiv preprint arXiv:2401.00368*, 2023.
- [47] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.
- [48] R. Meng, Y. Liu, S. R. Joty, C. Xiong, Y. Zhou, and S. Yavuz, "Sfr-embedding-mistral: enhance text retrieval with transfer learning." Salesforce AI Research Blog, 2024.
- [49] J. Kim, S. Lee, J. Kwon, S. Gu, Y. Kim, M. Cho, J. yong Sohn, and C. Choi, "Linq-embed-mistral: elevating text retrieval with improved gpt data through task-specific control and quality refinement." Linq AI Research Blog, 2024.
- [50] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "Llm2vec: Large language models are secretly powerful text encoders," *arXiv preprint arXiv:2404.05961*, 2024.
- [51] J. Ni, C. Qu, J. Lu, Z. Dai, G. H. Abrego, J. Ma, V. Zhao, Y. Luan, K. Hall, M.-W. Chang, *et al.*, "Large dual encoders are generalizable retrievers," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 9844–9855, 2022.
- [52] X. Li and J. Li, "Angle-optimized text embeddings," *arXiv preprint arXiv:2309.12871*, 2023.
- [53] D. Zhang, J. Li, Z. Zeng, and F. Wang, "Jasper and stella: distillation of sota embedding models," *arXiv preprint arXiv:2412.19048*, 2024.
- [54] A. Ng, "Sparse autoencoder," Tech. Rep. CS294A Lecture Notes, Stanford University, 2011.
- [55] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proceedings of the 25th International Conference on Machine Learning, ICML '08*, (New York, NY, USA), p. 1096–1103, Association for Computing Machinery, 2008.
- [56] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '17*, (New York, NY, USA), p. 665–674, Association for Computing Machinery, 2017.
- [57] S. A. Najafi, M. H. Asemani, and P. Setoodeh, "Attention and auto-encoder hybrid model for unsupervised online anomaly detection," 2024.
- [58] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "Lstm-based encoder-decoder for multi-sensor anomaly detection," 2016.
- [59] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in *Proceedings of the 35th International Conference on Machine Learning (J. Dy and A. Krause, eds.)*, vol. 80 of *Proceedings of Machine Learning Research*, pp. 4393–4402, PMLR, 10–15 Jul 2018.
- [60] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022.
- [61] I. Higgins, L. Matthey, A. Pal, C. P. Burgess, X. Glorot, M. M. Botvinick, S. Mohamed, and A. Lerchner, "beta-vae: Learning basic visual concepts with a constrained variational framework," in *International Conference on Learning Representations*, 2016.

- [62] N. Dilokthanakul, P. A. M. Mediano, M. Garnelo, M. C. H. Lee, H. Salimbeni, K. Arulkumaran, and M. Shanahan, "Deep unsupervised clustering with gaussian mixture variational autoencoders," 2017.
- [63] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019.
- [64] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Huggingface's transformers: State-of-the-art natural language processing," 2020.
- [65] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, A. Müller, J. Nothman, G. Louppe, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and Édouard Duchesnay, "Scikit-learn: Machine learning in python," 2018.

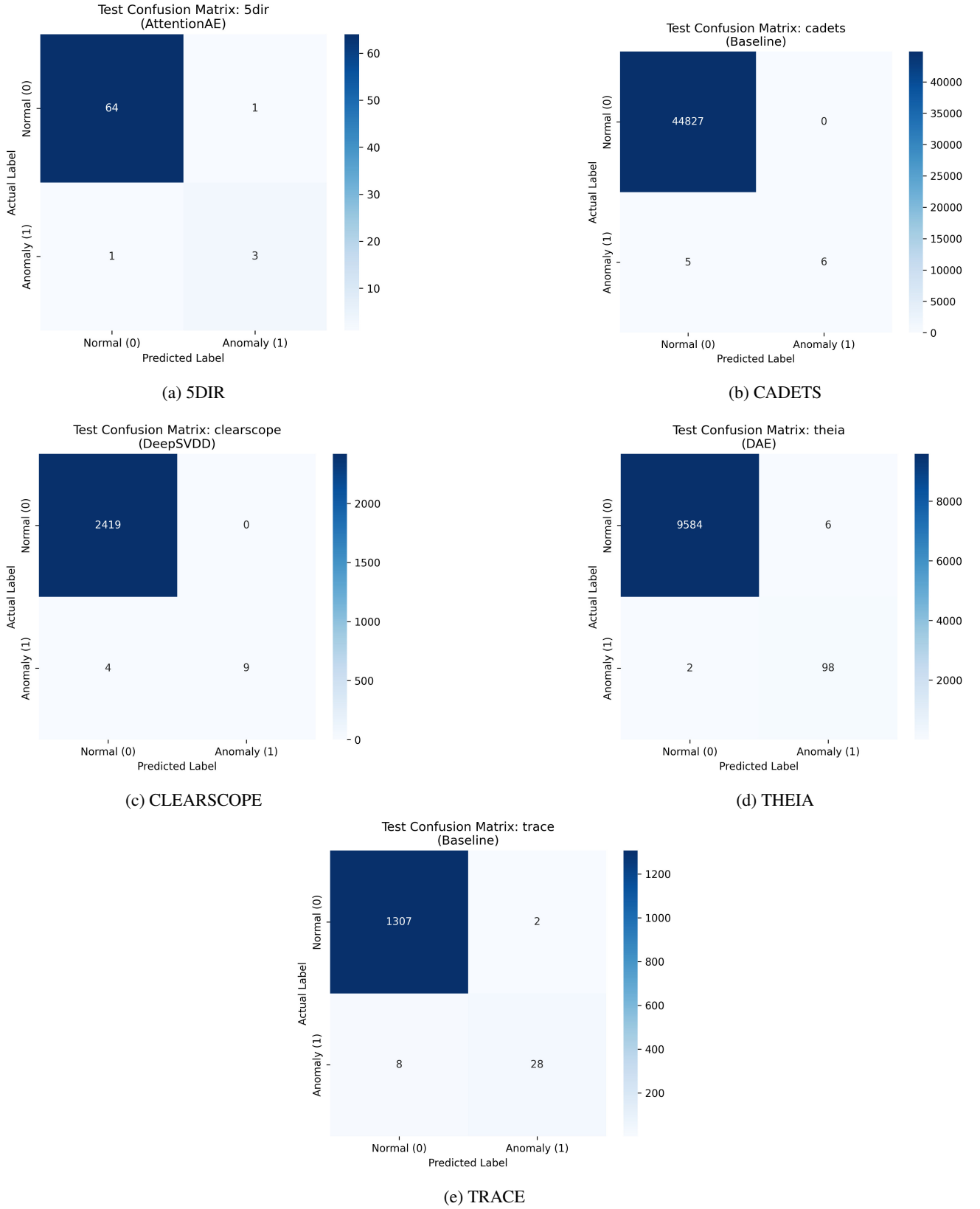


Figure 6: Best-performing configuration confusion matrices for each dataset.

XAI-Driven LLM-Based APT Anomaly Detection

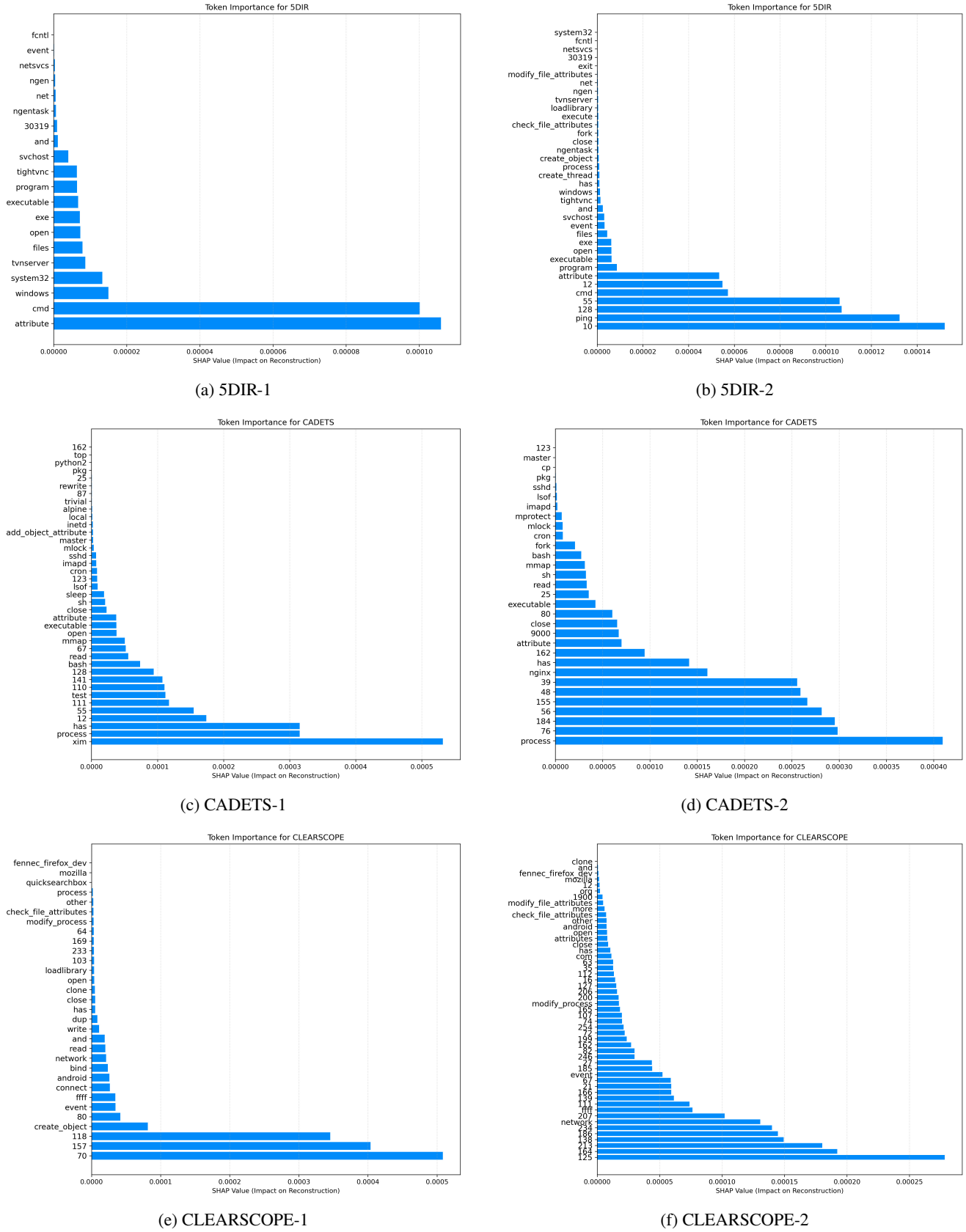


Figure 7: SHAP token attribution examples for 5DIR, CADETS, and CLEARSCOPE datasets.

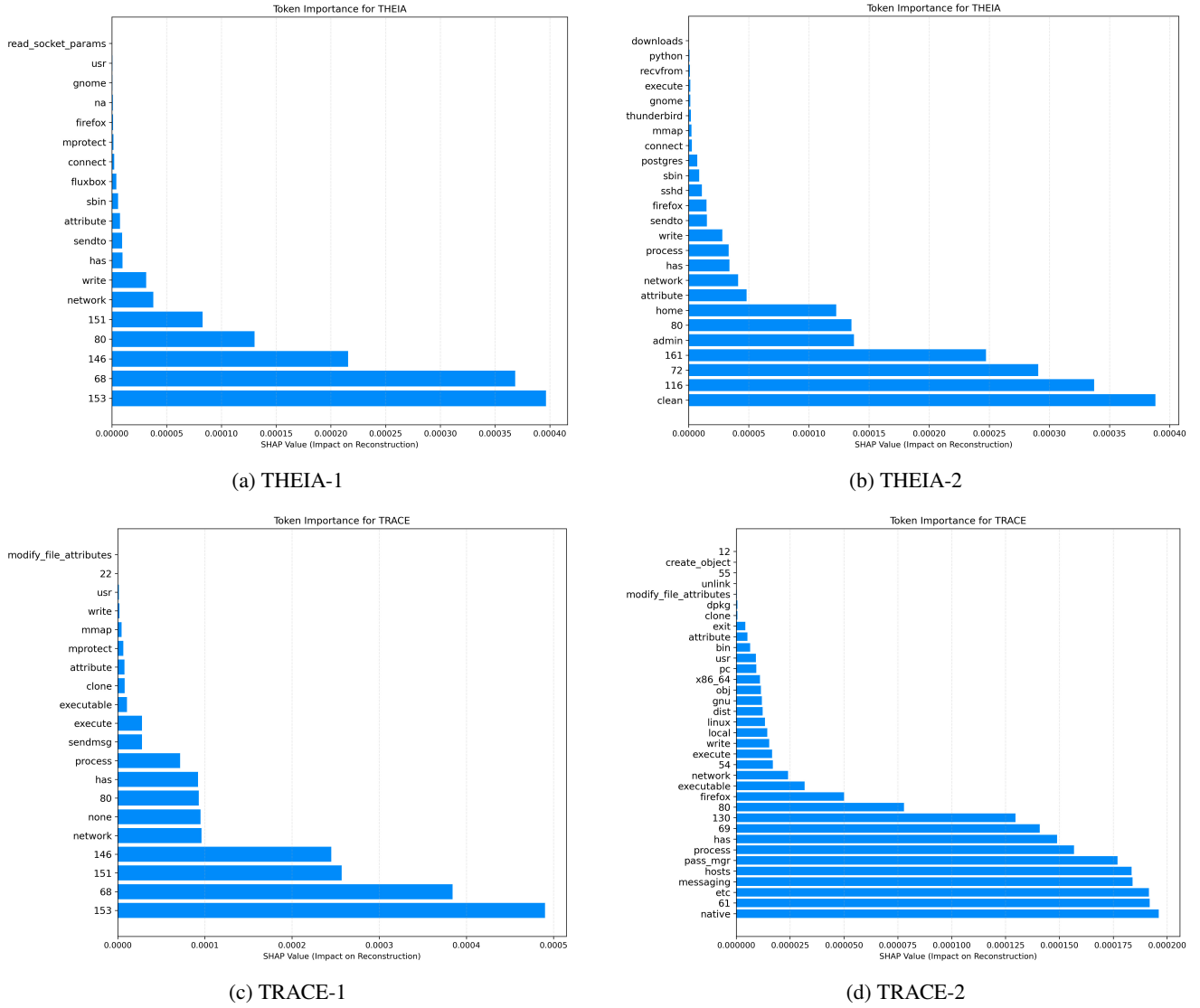


Figure 8: SHAP token attribution examples for THEIA and TRACE datasets.

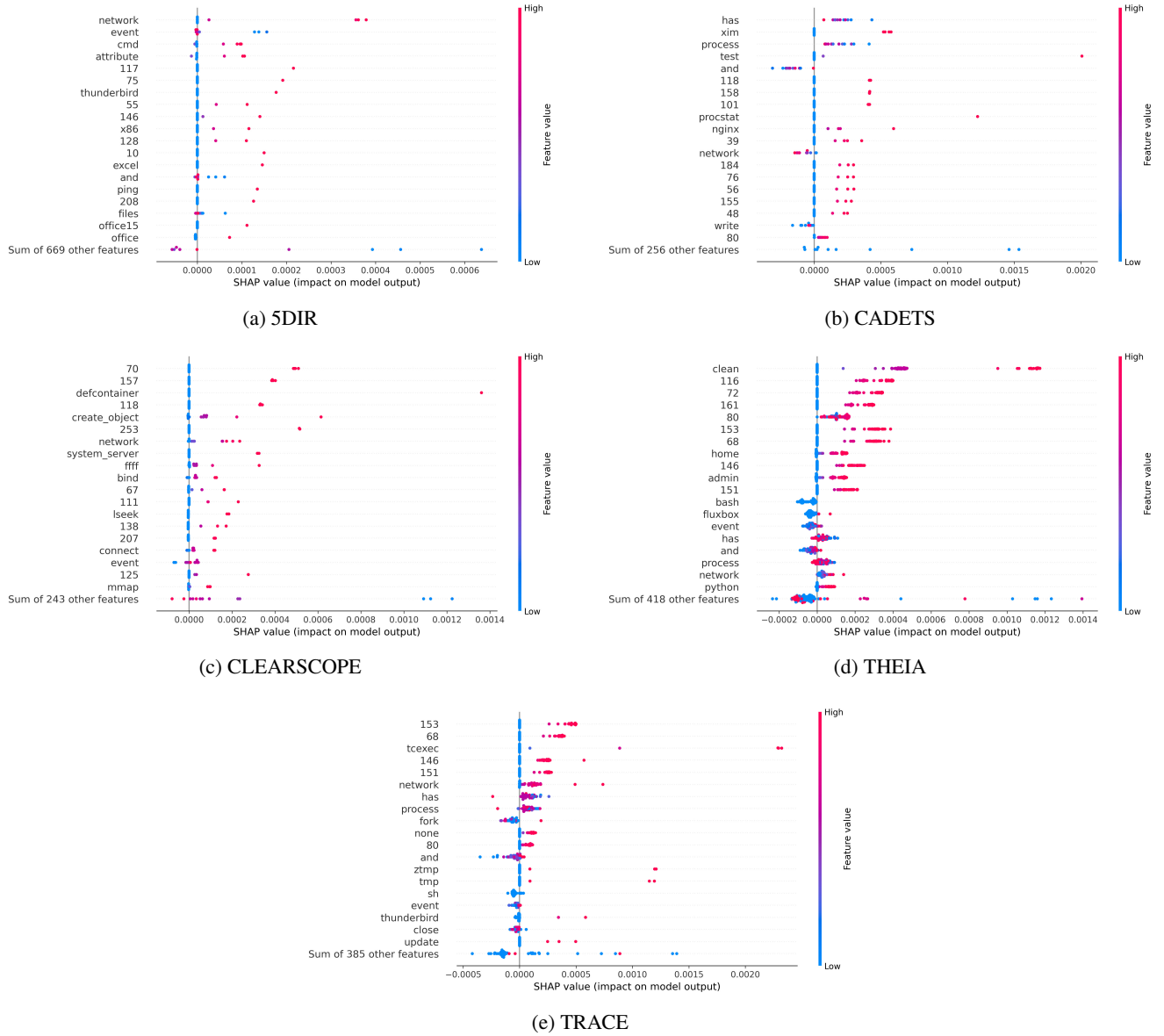


Figure 9: SHAP beeswarm plots across all datasets.